УТВЕРЖДАЮ	УТВЕРЖДАЮ
Руководитель по предмету «АСОИиУ»	студент гр. ИСИб-23-1 ИРНИТУ
_____ Д. С. Шипицин	_____ Л.А. Демонов
_____ 2026 г.	_____ 2026 г.
Печать	Печать

ИРНИТУ, институт ИТиАД

наименование организации - разработчика ТП на АС

Разработка мультиагентной системы ситуативного обучения с динамическим синтезом и верификацией адаптивных образовательных сценариев

наименование вида АС

Автономная цифровая среда интеллектуального тренинга

наименование объекта автоматизации

МАССО

сокращенное наименование АС

ТЕХНИЧЕСКИЙ ПРОЕКТ

На ___листах

Действует с «_____» _____ 2026 г.

СОГЛАСОВАНО

Руководитель

_____ Д. С. Шипицин

_____ 2026 г.

Печать

Содержание

1. Постановка проектных решений 3
2. Модель сущность-связь 5
3. Модели процессов To-Be 6
4. Диаграммы последовательности функциональностей 10
5. Модели данных 16
6. Архитектура системы в нотации C4 22
7. Модели информационных потоков данных 28
8. Шаблоны справочников и НСИ 30
9. Проектирование API 32
10. Модель структуры интерфейса 34
11. Макеты интерфейсов системы 35
12. Контрольные проектные сценарии 46
13. Вывод 47

1. Постановка проектных решений

Технический проект разработан для автоматизированной системы «МАССО» - мультиагентной системы ситуативного обучения с динамическим синтезом и верификацией адаптивных образовательных сценариев. Проектные решения синхронизированы с техническим заданием лабораторной работы 1: сохраняются роли пользователей, целевая технологическая база, трехагентная схема, гибридный LLM-контур, песочницы Docker/Kubernetes, автоматическая верификация и отчетность.

Объект проектирования - цифровая среда практического обучения, в которой обучающийся получает индивидуальный сценарий, работает в изолированной среде или документном тренажере, а система фиксирует цифровой след, выдает ограниченные подсказки, проверяет результат и обновляет граф компетенций.

Назначение проектных артефактов.

Артефакты технического проекта используются для перехода от требований к реализации: модель сущность-связь задает границы предметной области, To-Be модели уточняют работу модулей, диаграммы последовательности закрепляют взаимодействия, модели данных определяют хранилища, C4 фиксирует архитектуру, DFD описывает информационные потоки, а макеты показывают будущую структуру интерфейса без разработки готового приложения.

В таблице 1 приведена трассировка между требованиями ТЗ и решениями настоящего ТП.

Таблица 1 - Трассировка ТЗ и технического проекта

Требование ТЗ	**Проектное решение**	**Где раскрыто в ТП**	**Артефакты**
Ролевая модель и RBAC	JWT, refresh cookie, матрица прав и аудит изменения ролей.	Разделы 3, 8, 9, 10.	Рис. 2, Рис. 9, Рис. 42
Граф компетенций	Neo4j хранит навыки, зависимости, уровни и подтверждения обучающихся.	Разделы 2, 5, 7, 10.	Рис. 1, Рис. 22, Рис. 40
Генерация сценариев	ScenarioAgent формирует легенду, артефакты, checks и проверяет достижимость сценария.	Разделы 3, 4, 5, 8.	Рис. 4, Рис. 11
Учебные сессии и sandbox	Session API и Sandbox Manager создают изолированные среды с ограничениями ресурсов.	Разделы 3, 4, 5, 7.	Рис. 5, Рис. 12, Рис. 29
Подсказки и инциденты	AssessmentAgent ограничивает подсказки, фиксирует штраф и обрабатывает динамические события.	Разделы 3, 4, 8, 10.	Рис. 13, Рис. 36

Автоматическая проверка	Verification Engine запускает технические и документные checks, сохраняет результат и обновляет профиль.	Разделы 3, 4, 5, 8.	Рис. 6, Рис. 14
Отчеты и экспертная оценка	Reports API поддерживает периодные фильтры, экспорт и ручную корректировку с причиной.	Разделы 3, 8, 9, 10.	Рис. 7, Рис. 15, Рис. 18
Гибридный LLM-контур	LLM Gateway переключает внешний, локальный и шаблонный режимы без остановки активных сессий.	Разделы 4, 7, 8, 10.	Рис. 17, Рис. 43

2. Модель сущность-связь

Модель сущность-связь определяет основные объекты MACCO: пользователей, роли, компетенции, сценарии, сессии, события, подсказки, результаты проверки, отчеты и эксплуатационные настройки. Центральной операционной сущностью является учебная сессия: она связывает обучающегося, индивидуальный запуск сценария, цифровой след, проверки и итоговый отчет.

ER-модель представлена на Рис. 1. Рисунок показывает, что справочные данные отделены от операционных: роли, предметные области, типы сценариев, политики подсказок и sandbox-профили ведутся отдельно, а события сессии и результаты проверки имеют обязательные корреляционные идентификаторы.

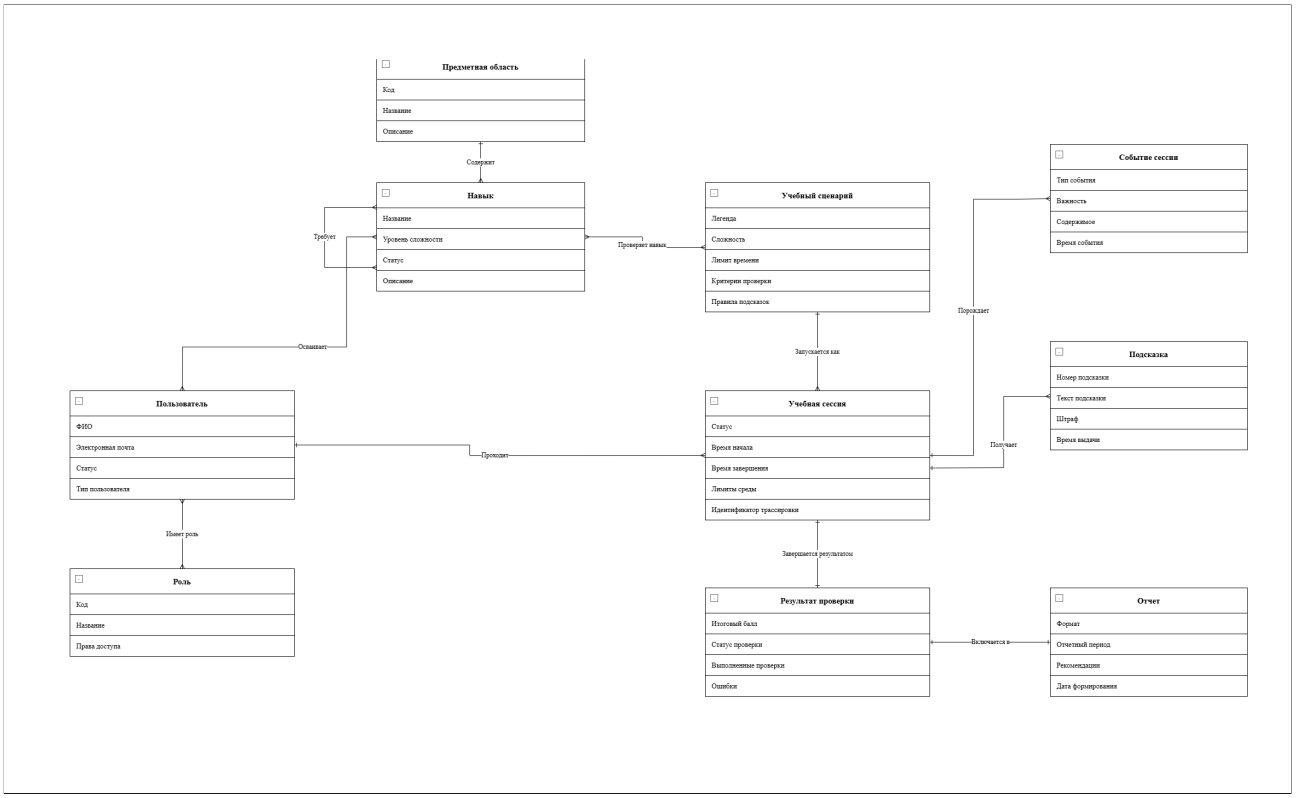


Рис. 1. Модель сущность-связь MACCO

Связи модели:

- один пользователь может иметь несколько ролей через связующую сущность `UserRole`;
- один шаблон сценария имеет несколько индивидуальных запусков `ScenarioRun`;
- один запуск сценария порождает одну или несколько учебных сессий;
- одна учебная сессия содержит события, подсказки, проверки и итоговый отчет;
- компетенции и сценарии связаны отношением `TARGETS`, а пользователь и компетенция - отношением `HAS_SKILL`;
- изменения оценок, ролей, LLM-режима и sandbox-политик фиксируются в журнале аудита.

3. Модели процессов To-Be

Процессы To-Be описывают целевую работу каждого ключевого модуля системы. В отличие от ТЗ, где сформулированы требования, здесь зафиксирован проектный порядок обработки данных, ответственности модулей и точки сохранения состояния.

Процесс аутентификации и разграничения прав приведен на Рис. 2. Пользователь проходит вход, Auth/RBAC проверяет учетные данные и права, затем фронтенд открывает только те разделы, которые разрешены назначенными ролями.

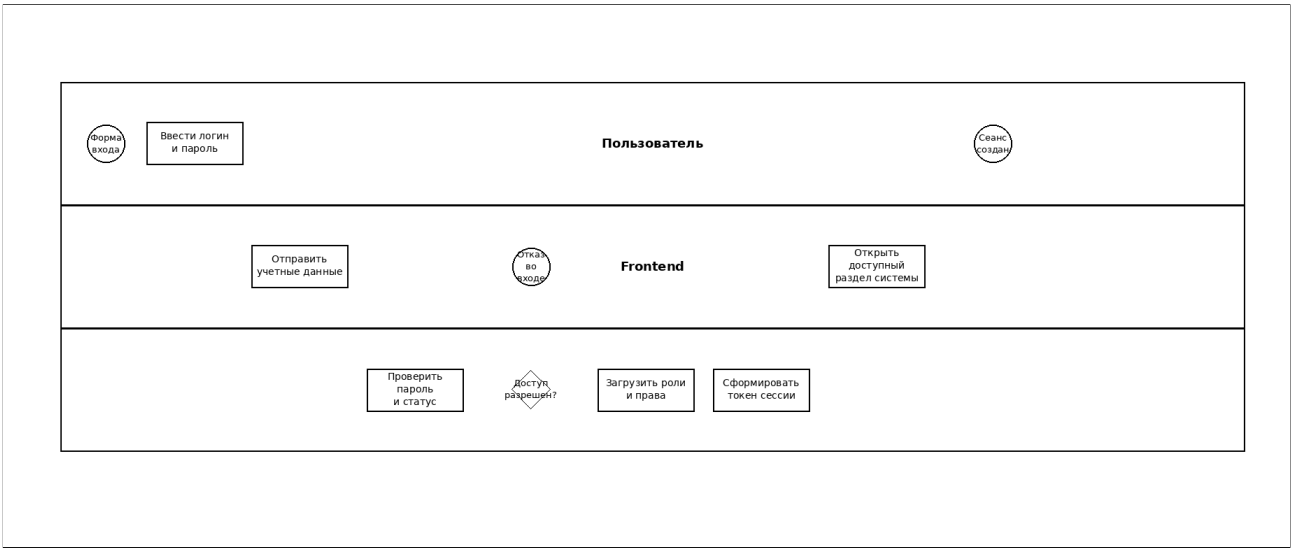


Рис. 2. To-Be процесс аутентификации и RBAC

Процесс формирования профиля компетенций приведен на Рис. 3. ProfileAgent загружает граф из Neo4j, вычисляет дефициты навыков и возвращает рекомендации по сценариям.

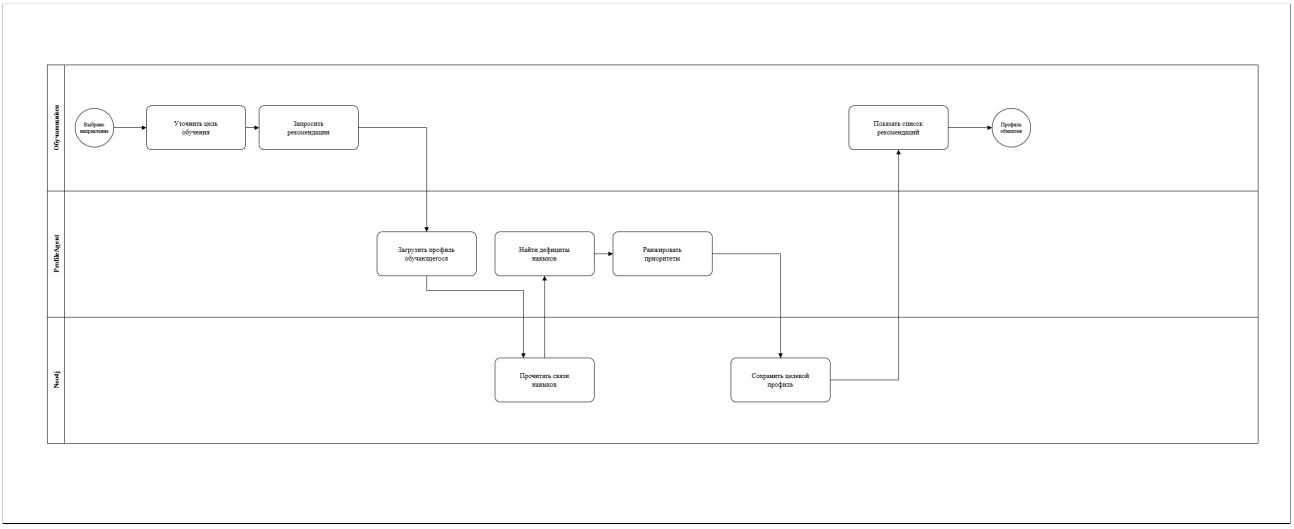


Рис. 3. To-Be процесс формирования профиля компетенций

Процесс генерации сценария приведен на Рис. 4. ScenarioAgent использует шаблоны, контекст из ChromaDB и LLM Gateway; до публикации сценарий проверяется на достижимость, непротиворечивость и безопасность.

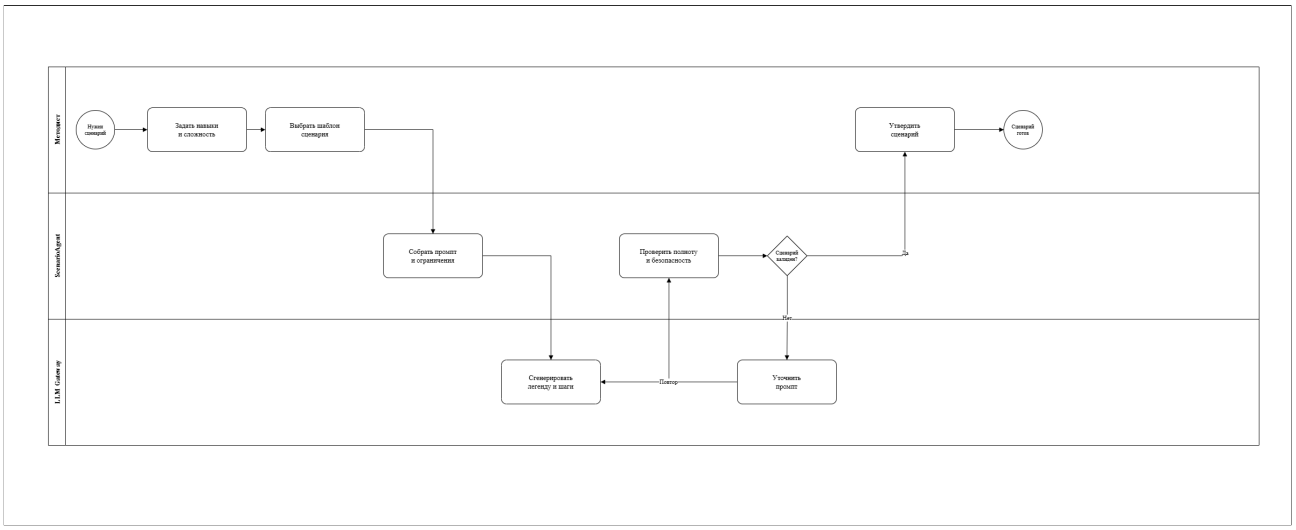


Рис. 4. To-Be процесс генерации и валидации сценария

Процесс запуска учебной среды приведен на Рис. 5. Session API создает сессию, Sandbox Manager разворачивает контейнеры или документный тренажер, Redis публикует статусы, а фронтенд открывает рабочее место.

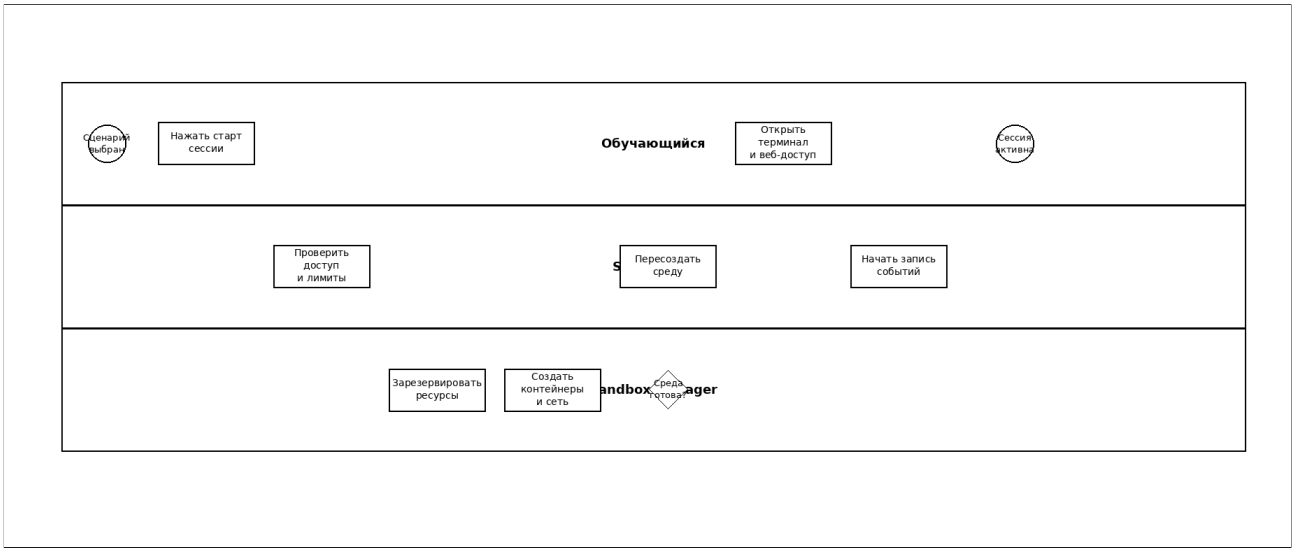


Рис. 5. To-Be процесс запуска учебной среды

Процесс сопровождения и проверки приведен на Рис. 6. AssessmentAgent анализирует события, выдает не более трех подсказок, запускает Verification Engine и передает результаты в PostgreSQL и Neo4j.

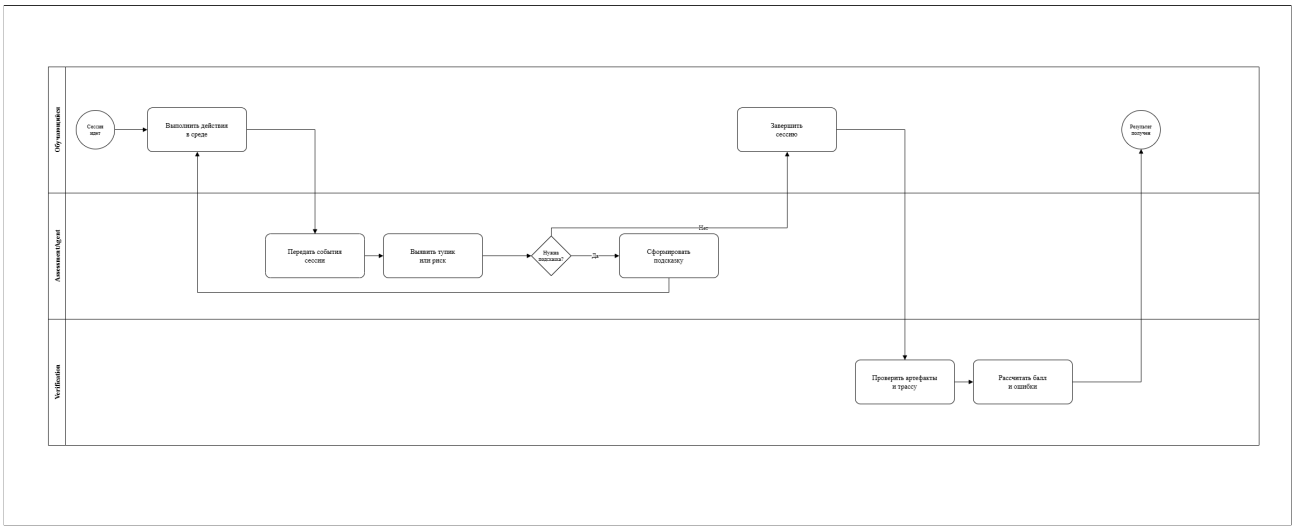


Рис. 6. To-Be процесс сопровождения и автоматической проверки

Процесс отчетности и экспертной корректировки приведен на Рис. 7. Преподаватель использует хронологические фильтры, анализирует цифровой след и при необходимости корректирует оценку с обязательным комментарием.

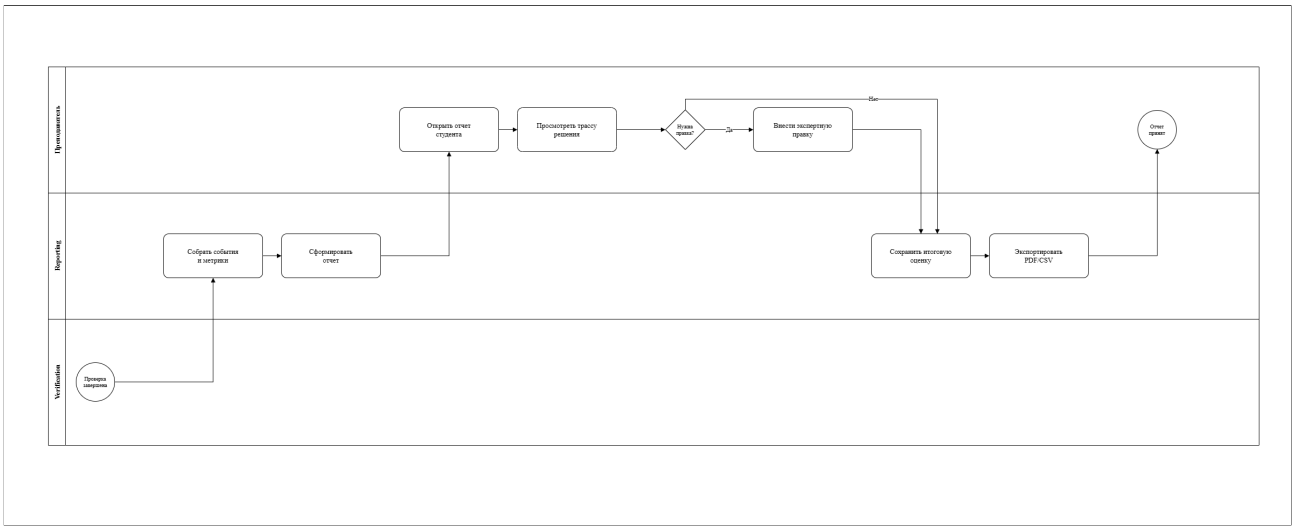


Рис. 7. To-Be процесс отчетности и экспертной корректировки

Процесс администрирования и мониторинга приведен на Рис. 8. Администратор управляет пользователями, LLM-провайдерами, sandbox-профилями, очередями и аудитом.

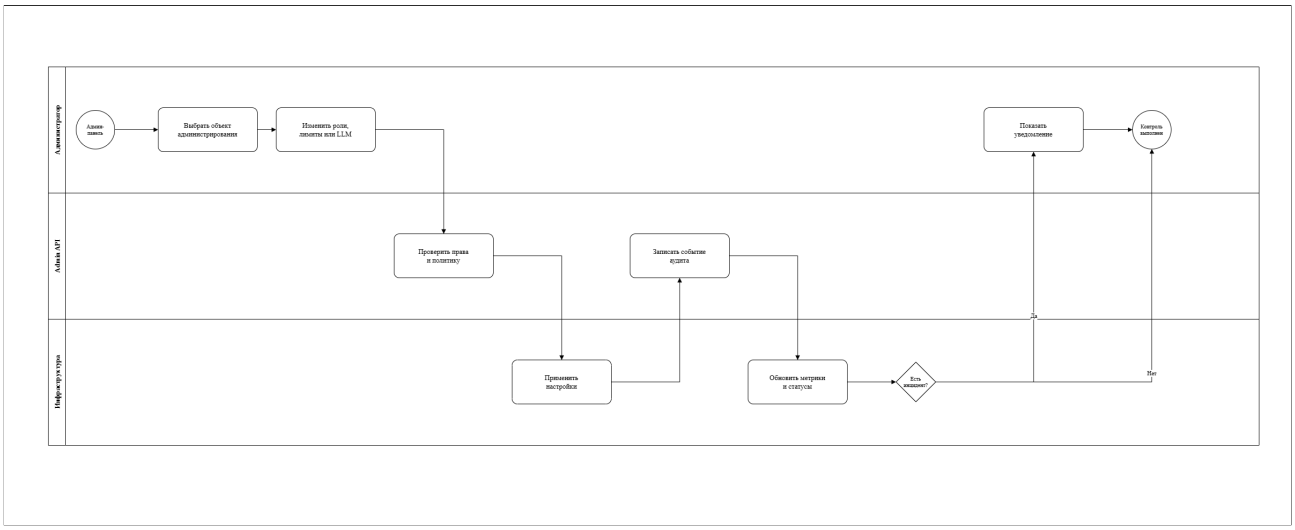


Рис. 8. To-Be процесс администрирования и мониторинга

4. Диаграммы последовательности функциональностей

Диаграммы последовательности уточняют вызовы между пользователем, фронтендом, API, агентами, хранилищами и инфраструктурой. Каждая диаграмма соответствует функциональности из ТЗ и закрепляет, какие данные передаются между участниками.

Последовательность входа пользователя показана на Рис. 9. После проверки учетных данных система возвращает access-токен, refresh cookie и стартовый маршрут по роли.

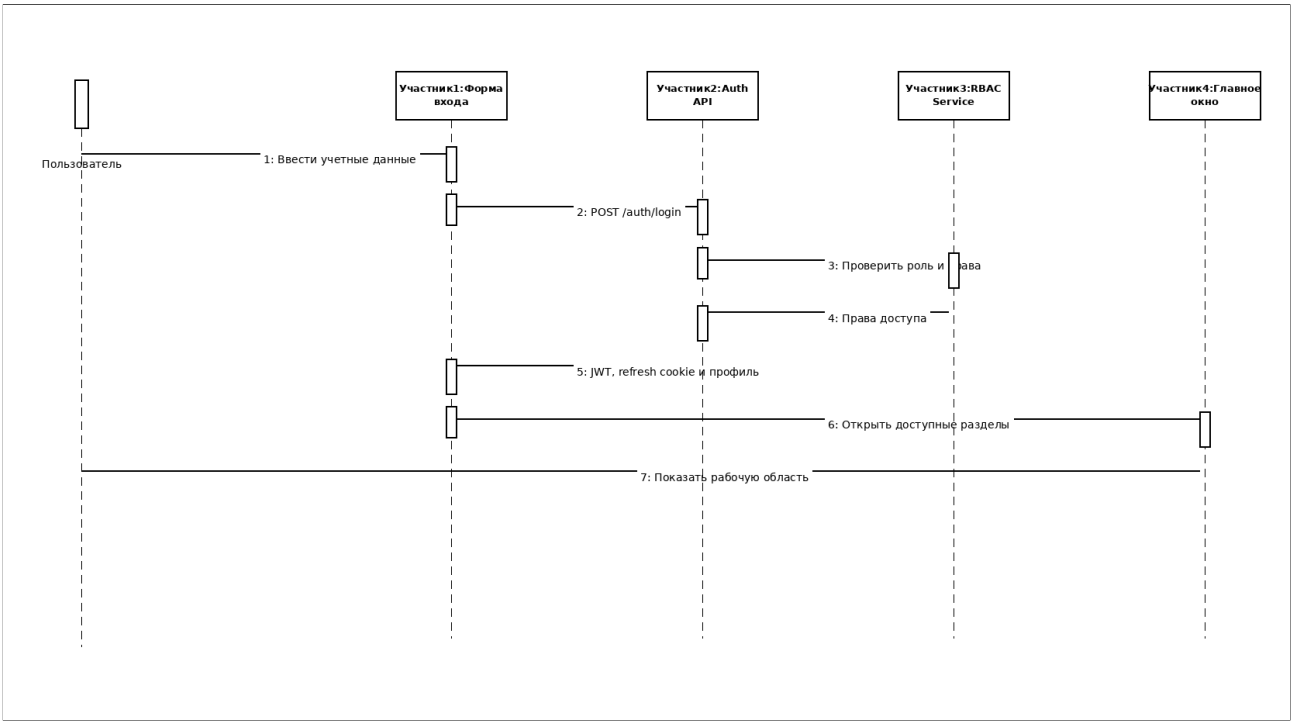


Рис. 9. Диаграмма последовательности входа пользователя

Подбор сценария по профилю компетенций показан на Рис. 10. Вызов начинается с запроса рекомендаций и завершается списком дефицитов навыков с приоритетами.

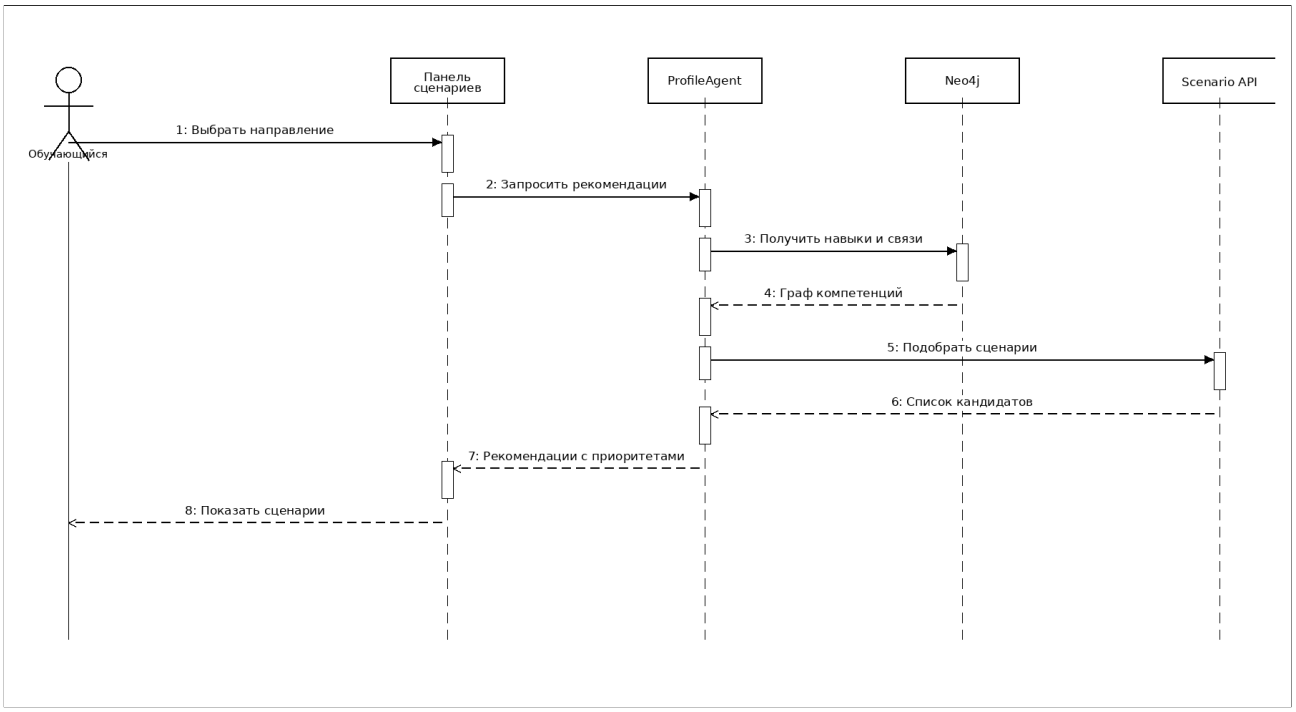


Рис. 10. Диаграмма последовательности подбора сценария по профилю

Генерация сценария показана на Рис. 11. ScenarioAgent обращается к ChromaDB для запрета повторов, к LLM Gateway для генерации и к PostgreSQL для сохранения утвержденного шаблона.

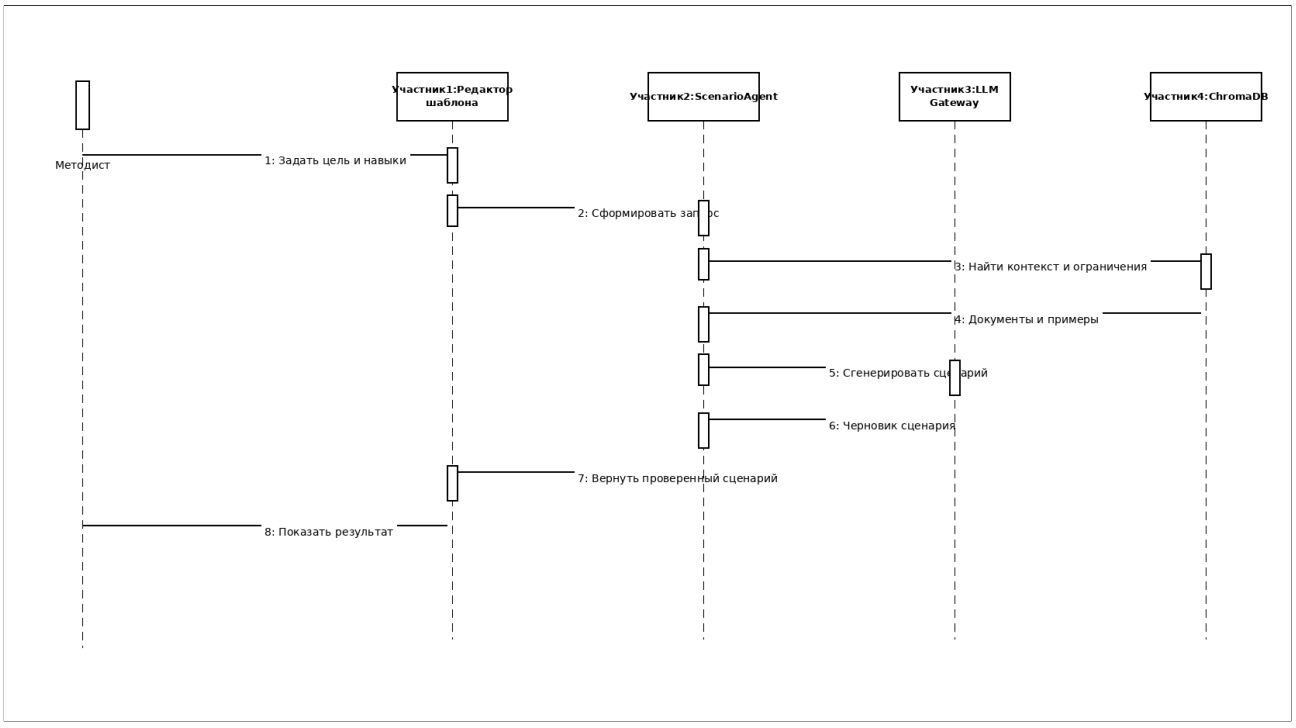


Рис. 11. Диаграмма последовательности генерации сценария

Запуск учебной сессии показан на Рис. 12. Backend создает `session_id`, Sandbox Manager поднимает среду, Redis публикует статус готовности, а фронтенд открывает WebSocket-канал.

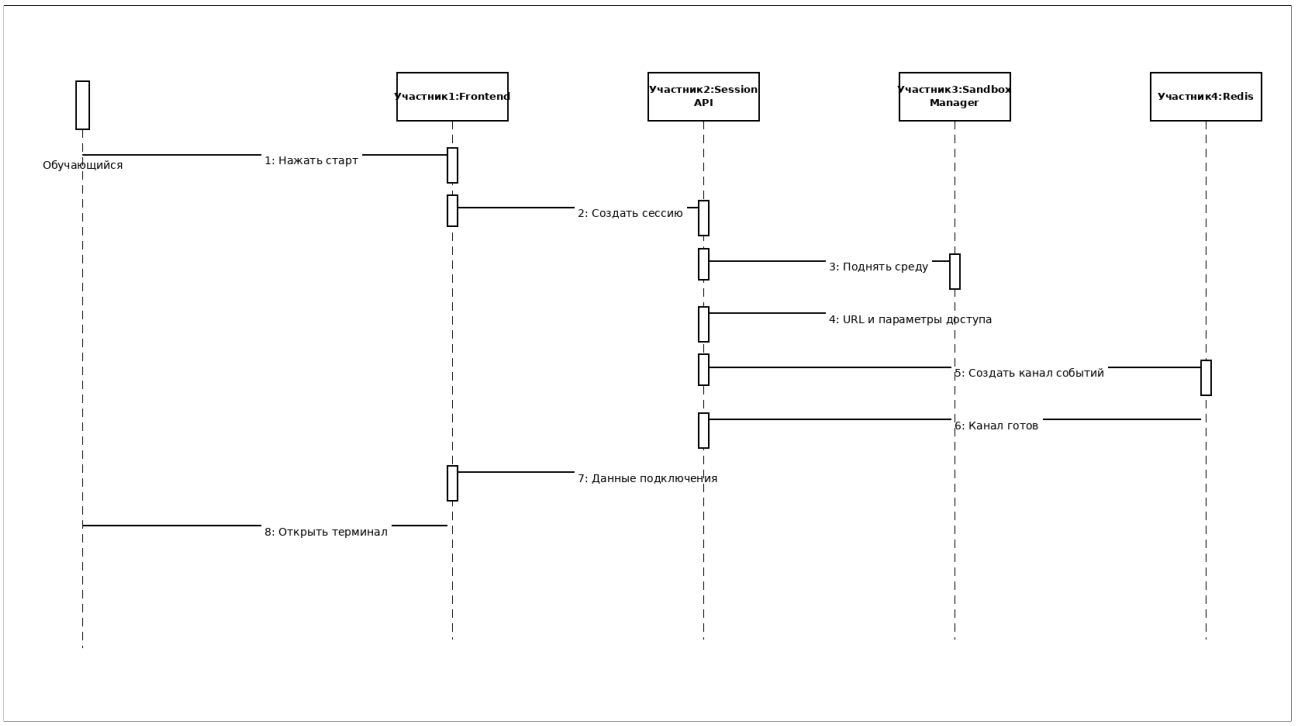


Рис. 12. Диаграмма последовательности запуска учебной сессии

Внесение инцидента и выдача подсказки показаны на Рис. 13. Инцидент отображается обучающемуся как событие состояния, а подсказка сохраняется с номером и штрафом.

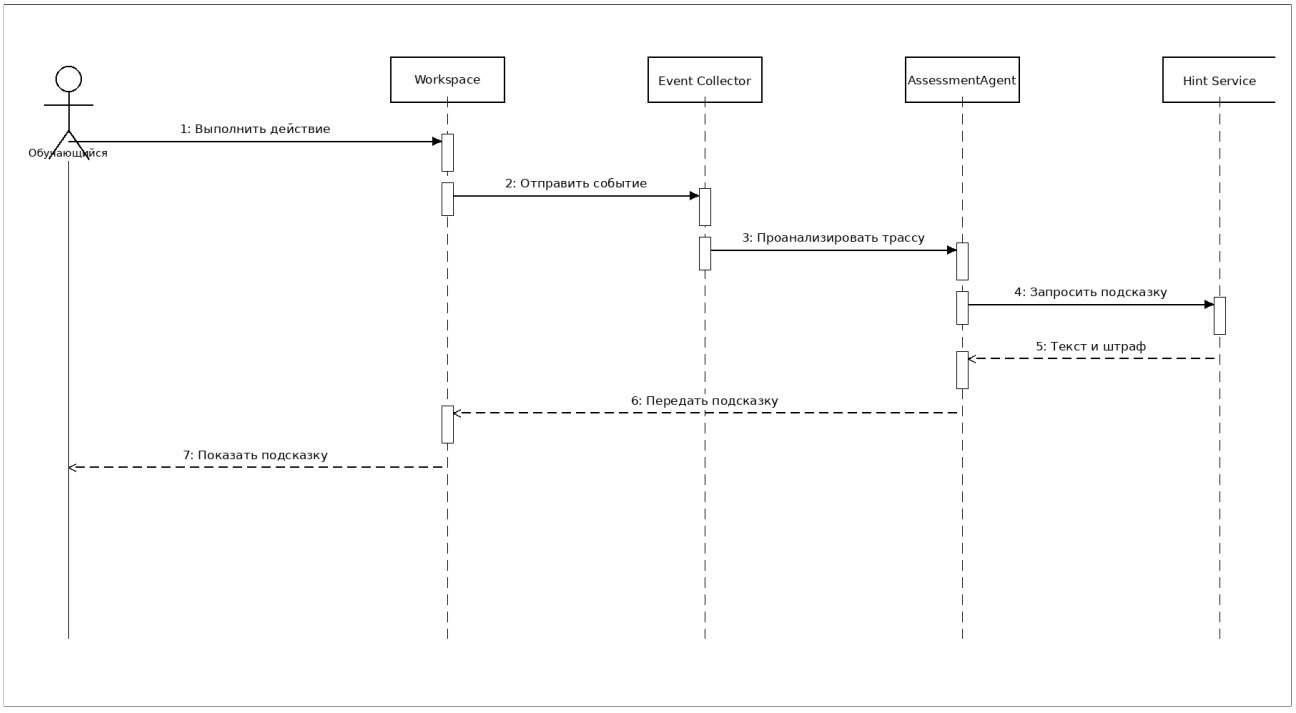


Рис. 13. Диаграмма последовательности обработки инцидента и подсказки

Проверка результата и формирование отчета показаны на Рис. 14. Verification Engine запускает checks, AssessmentAgent рассчитывает балл, Reports API формирует отчет.

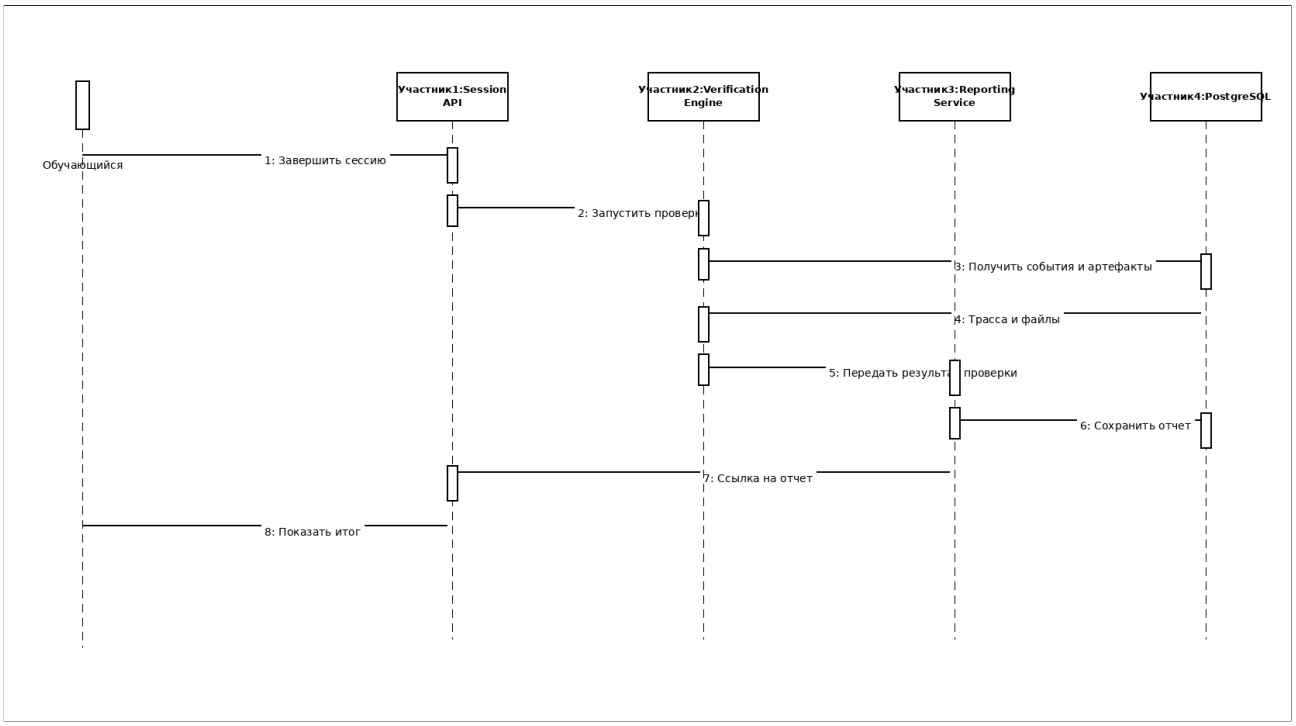


Рис. 14. Диаграмма последовательности проверки результата и отчета

Экспертная проверка преподавателем показана на Рис. 15. При корректировке оценки причина изменения обязательна и попадает в `audit_logs`.

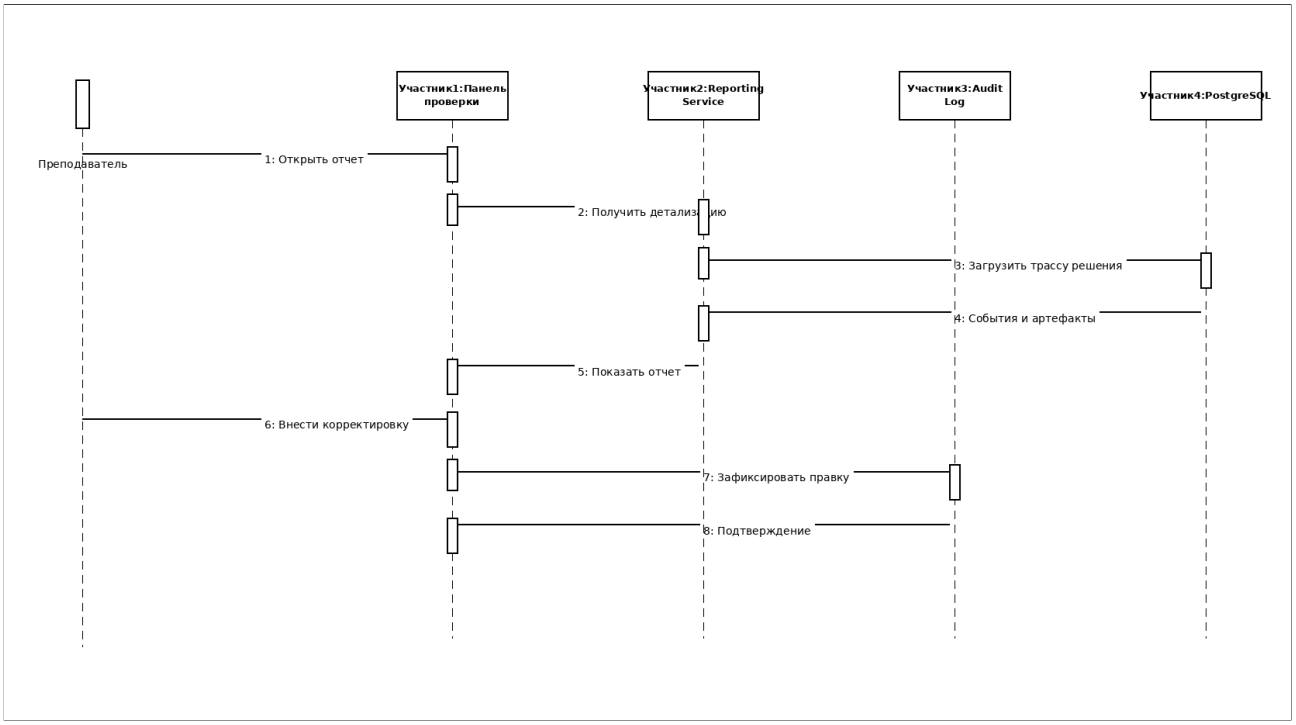


Рис. 15. Диаграмма последовательности экспертной проверки преподавателем

Создание шаблона сценария методистом показано на Рис. 16. Методист задает навык, связи prerequisite, checks, лимиты и правила подсказок.

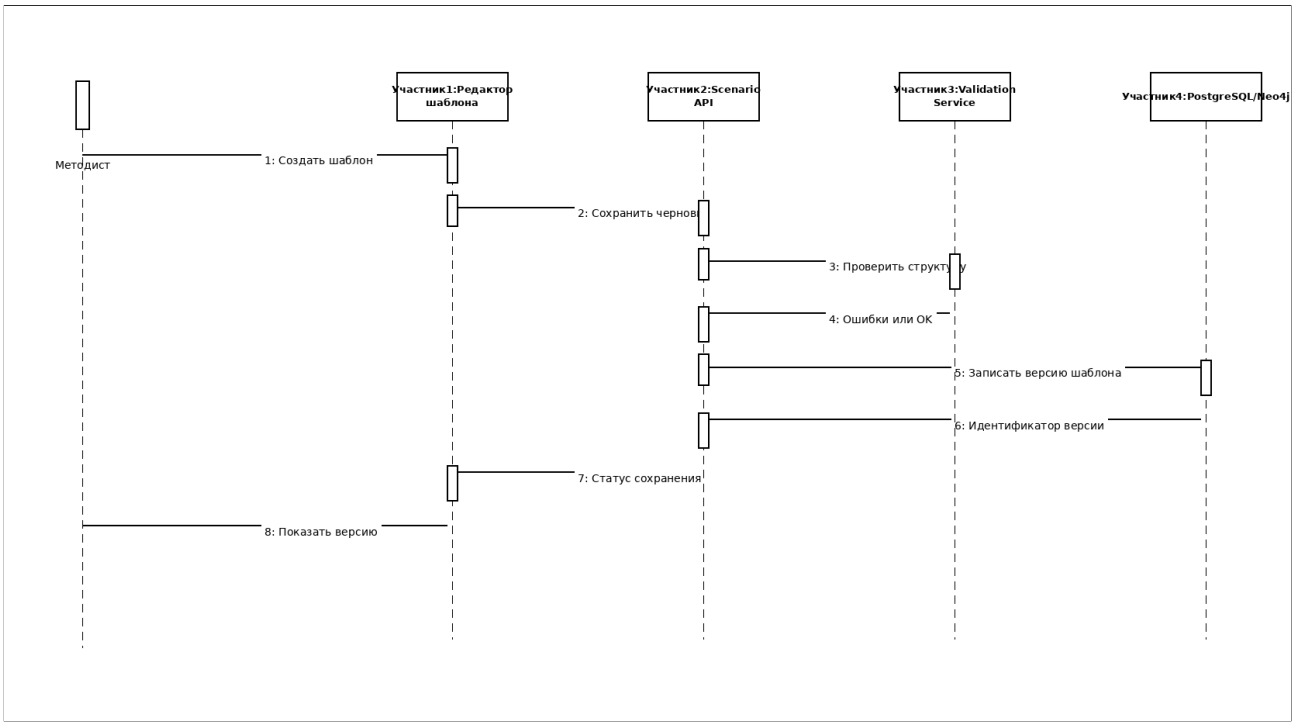


Рис. 16. Диаграмма последовательности создания шаблона сценария

Переключение LLM-контура показано на Рис. 17. Новые задачи переводятся на выбранный режим, активные учебные сессии не останавливаются.

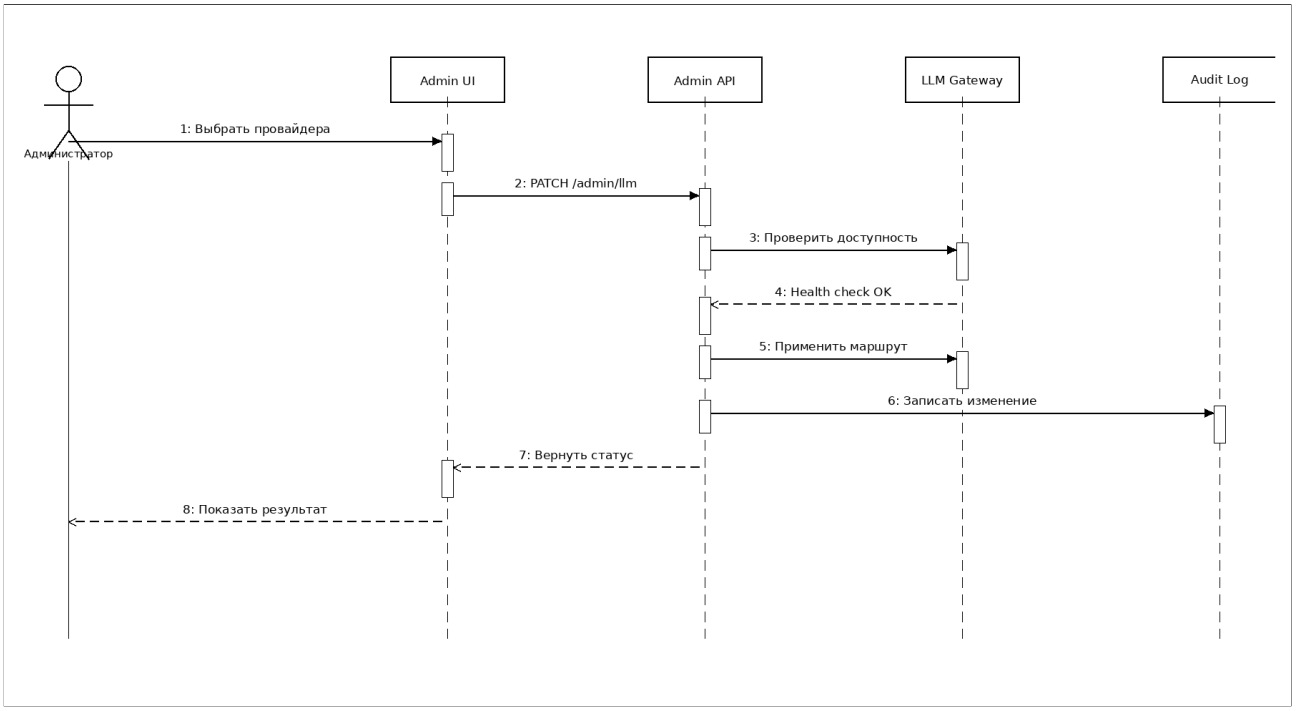


Рис. 17. Диаграмма последовательности переключения LLM-контура

Экспорт отчетности показан на Рис. 18. Обязательными входными параметрами являются начало и конец периода, а также формат выгрузки.

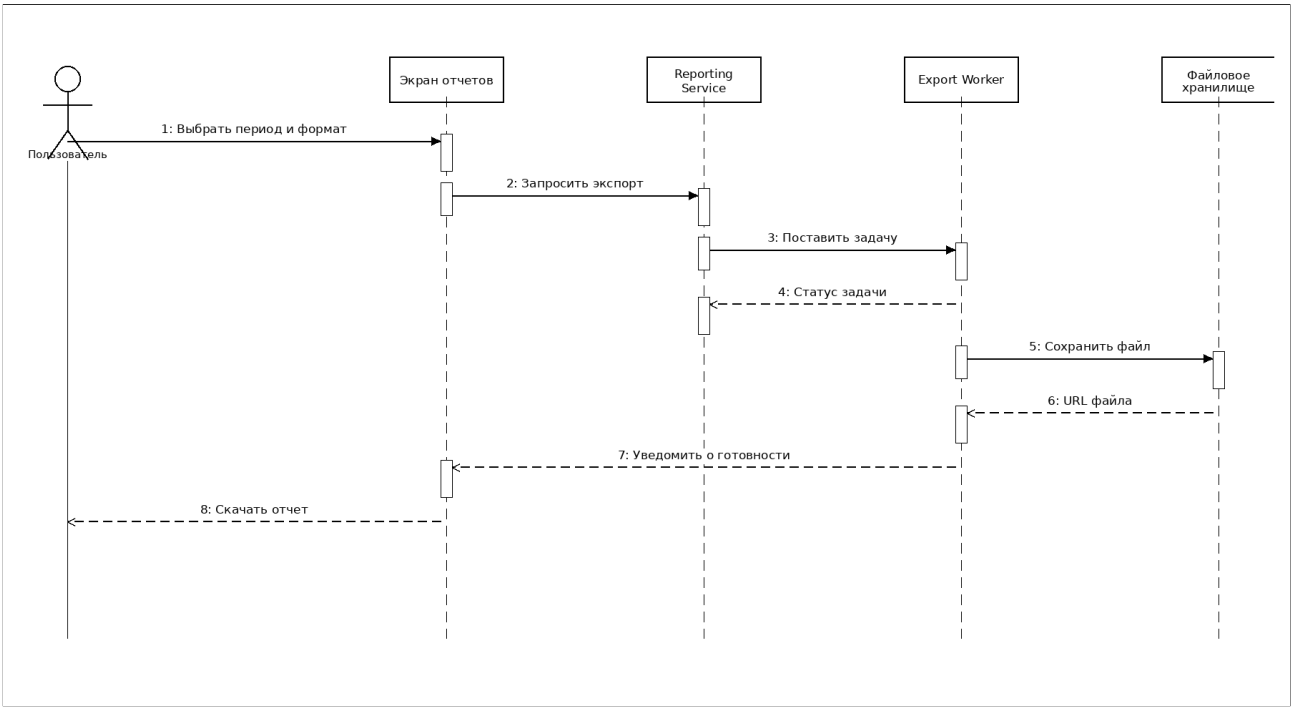


Рис. 18. Диаграмма последовательности экспорта отчетности

5. Модели данных

Модели данных представлены в четырех уровнях: концептуальном, логическом и физическом. Физический уровень разделен по СУБД и хранилищам, так как ТЗ фиксирует PostgreSQL, Neo4j, ChromaDB и Redis как разные контуры данных.

Концептуальная модель на Рис. 19 показывает крупные классы данных: субъекты, обучение, оценивание и эксплуатацию.

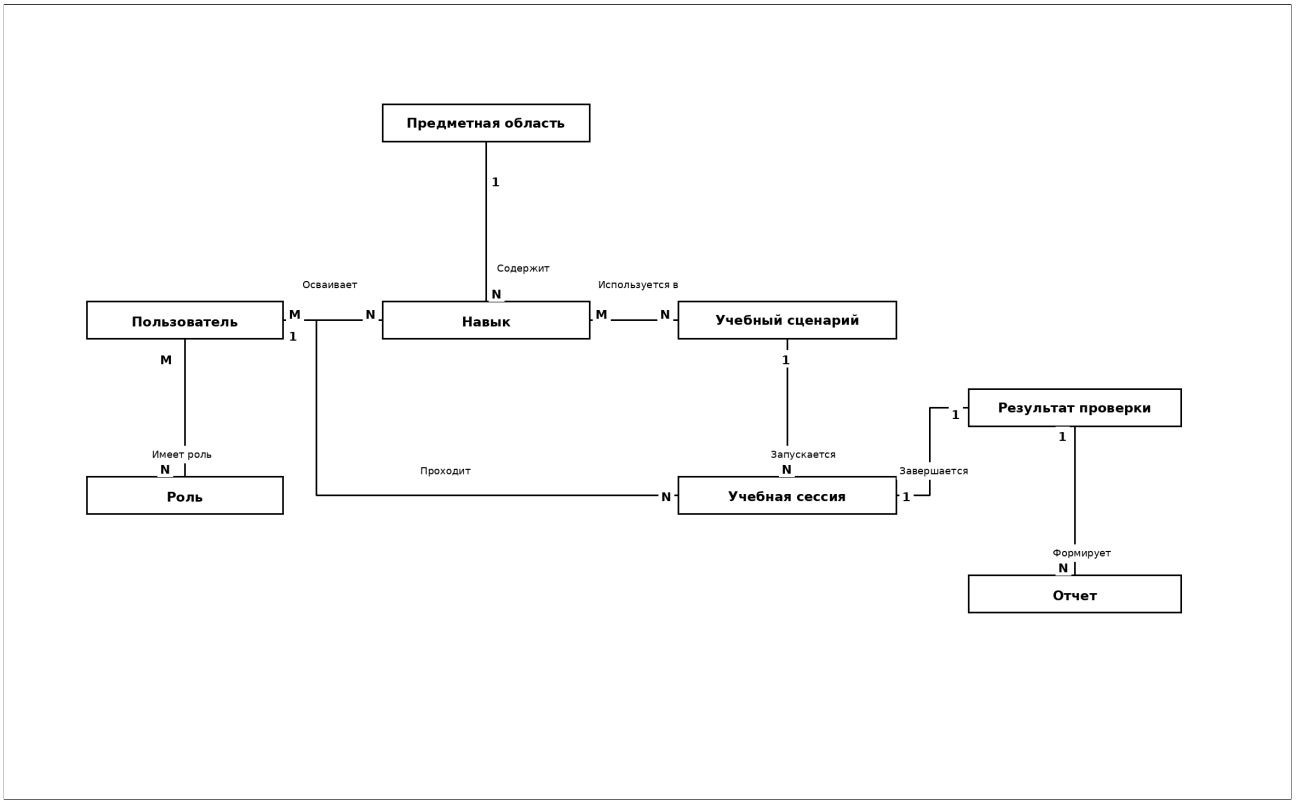


Рис. 19. Концептуальная модель данных

Логическая модель на Рис. 20 показывает состав сущностей, ключей и связей, которые затем уточняются в физических моделях для выбранных хранилищ.

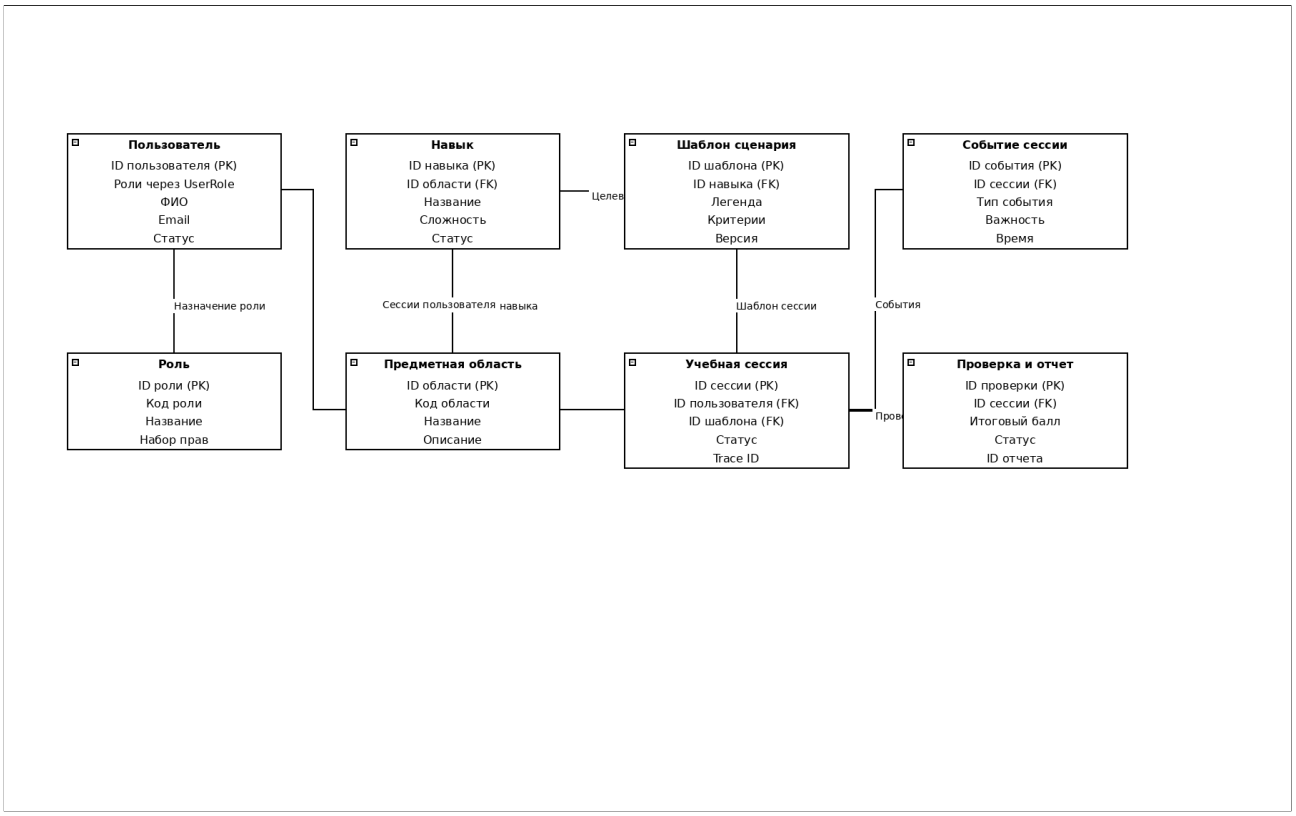


Рис. 20. Логическая модель данных

Физическая модель PostgreSQL приведена на Рис. 21. Основные таблицы: `users`, `roles`, `user_roles`, `scenario_templates`, `scenario_runs`, `learning_sessions`, `session_events`, `hints`, `verification_results`, `reports`, `llm_providers`, `sandbox_profiles`, `audit_logs`.

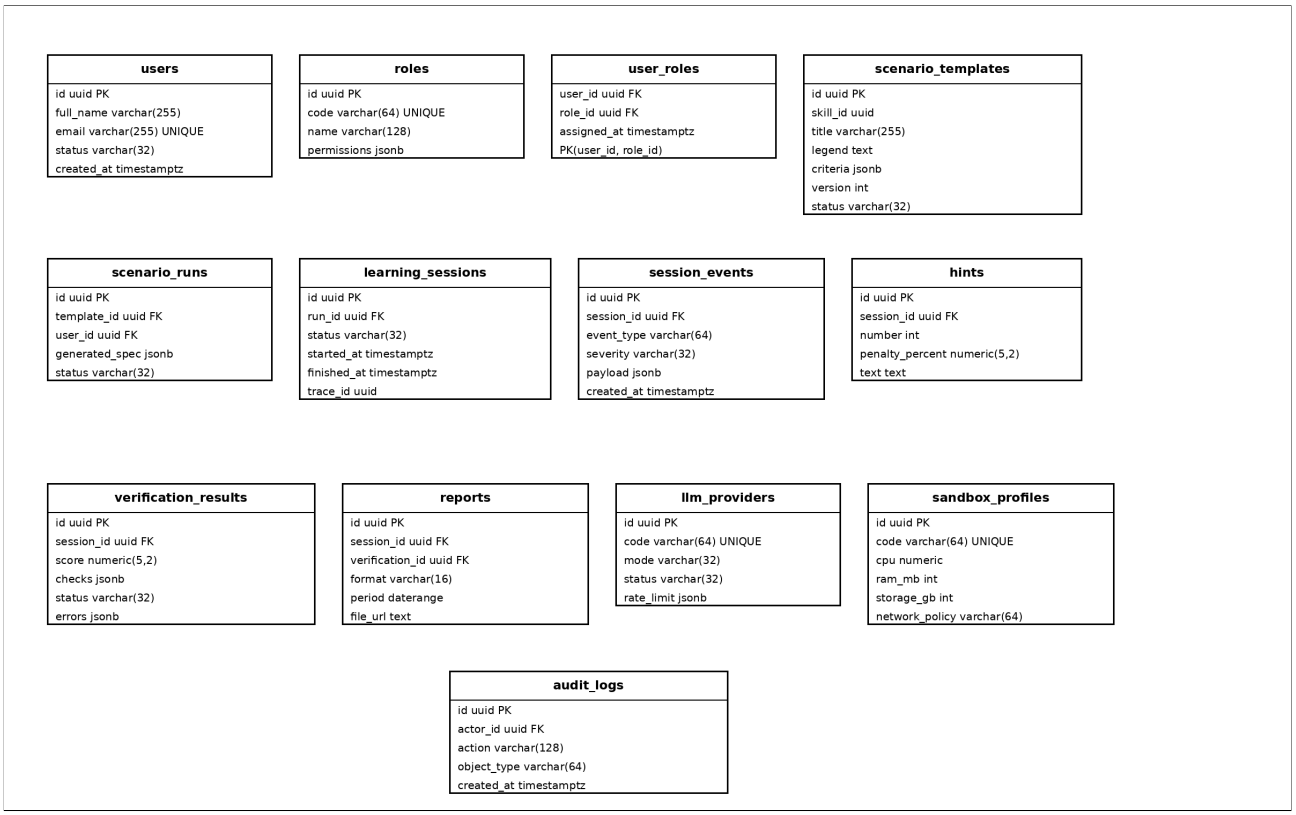


Рис. 21. Физическая модель данных PostgreSQL

Физическая модель Neo4j приведена на Рис. 22. Графовые узлы и связи соответствуют требованиям ТЗ: `User`, `Skill`, `Domain`, `Scenario`, `HAS_SKILL`, `REQUIRES`, `TARGETS`, `BELONGS_TO`.

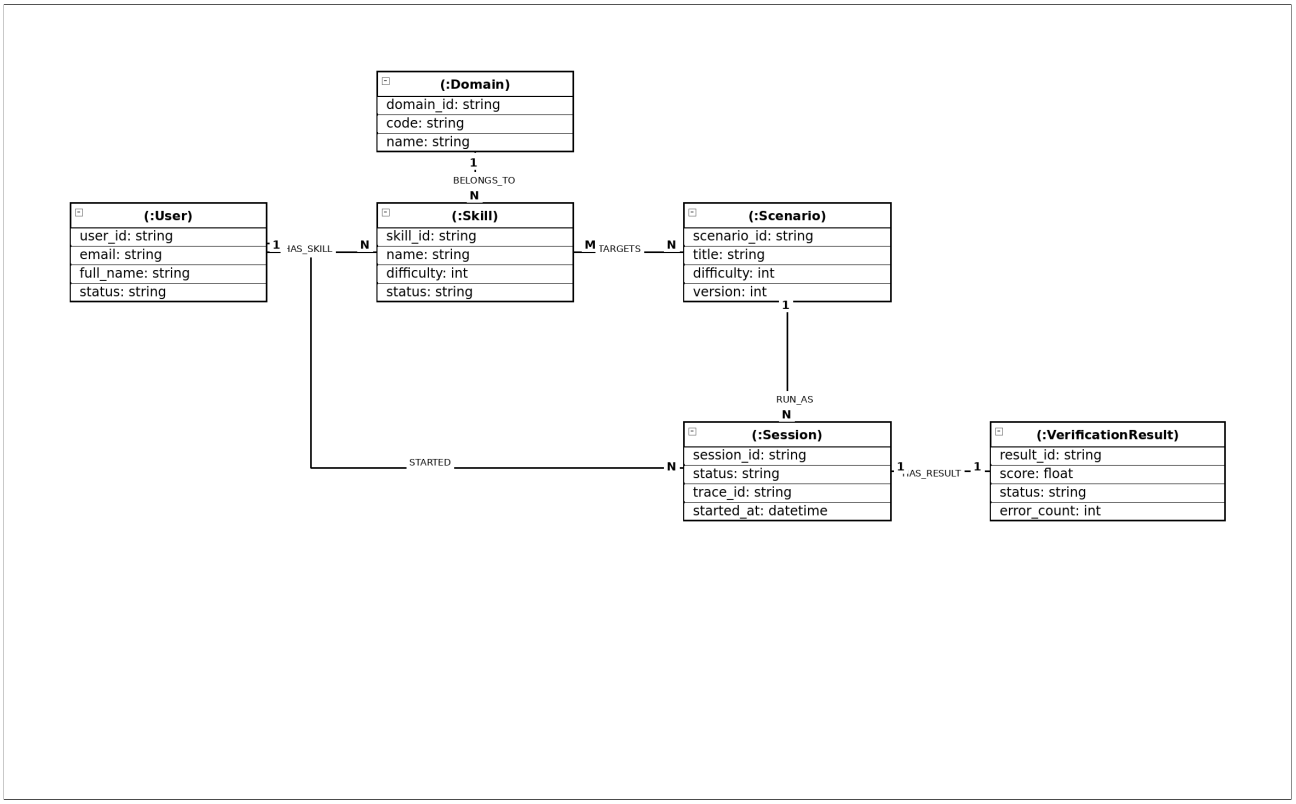


Рис. 22. Физическая модель данных Neo4j

Физическая модель ChromaDB приведена на Рис. 23. Векторные коллекции используются для легенд, документов, подсказок и подтвержденных альтернативных решений.

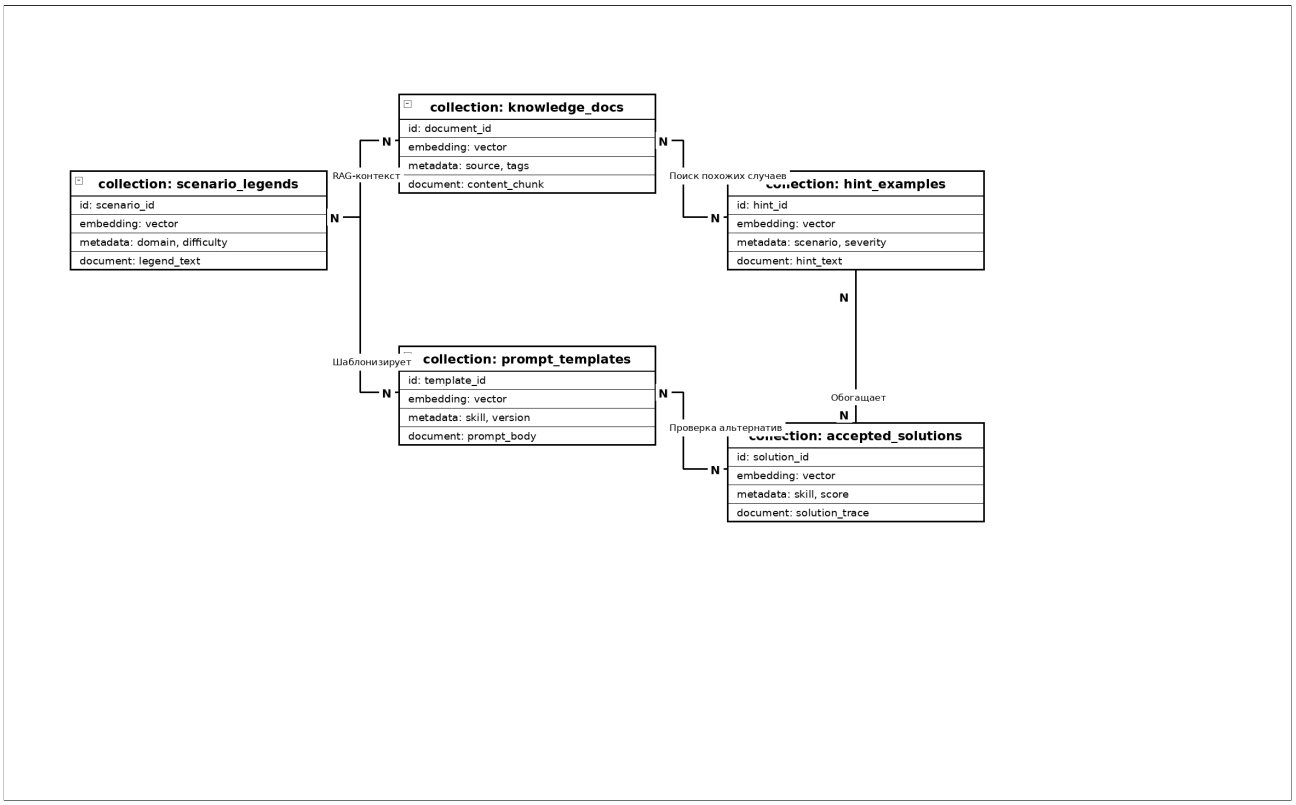


Рис. 23. Физическая модель данных ChromaDB

Физическая модель Redis приведена на Рис. 24. Redis хранит очереди, статусы, rate limit, счетчики подсказок и сообщения Pub/Sub.

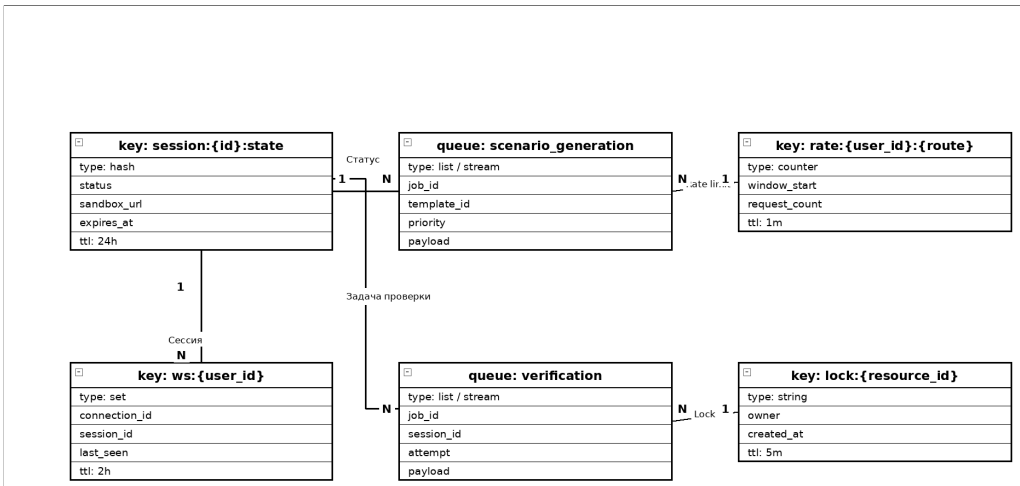


Рис. 24. Физическая модель данных Redis

Таблица 2 - Основные сущности API и данных

Сущность	Назначение	Ключевые поля	Хранилище

User	Учетная запись пользователя.	user_id, email, full_name, status, created_at	PostgreSQL
Role	Роль доступа.	role_id, code, name, description	PostgreSQL
Skill	Компетенция предметной области.	skill_id: string, name: string, difficulty: int, status: string	Neo4j
Competency Graph	Навыки и prerequisite-связи.	HAS_SKILL.level: int, TARGETS.weight: float, last_confirmed: datetime	Neo4j
Scenario Template	Утвержденный шаблон сценария.	template_id, version, domain, spec, status	PostgreSQL
Scenario Run	Индивидуальный запуск сценария.	run_id, template_id, user_id, generated_spec	PostgreSQL
Learning Session	Факт выполнения задания.	session_id, run_id, status, started_at, finished_at	PostgreSQL
Session Event	Цифровой след действий.	event_id, session_id, type, severity, payload, trace_id	PostgreSQL
Hint	Ограниченная подсказка.	hint_id, session_id, number, penalty_percent, text	PostgreSQL
Verification Result	Результат автоматической проверки.	result_id, session_id, score, checks, status	PostgreSQL
Report	Итоговый отчет и экспорт.	report_id, session_id, format, period_from, period_to	PostgreSQL
Llm Provider	Провайдер генерации и резервный режим.	provider_id, code, mode, status, rate_limit	PostgreSQL
Sandbox Profile	Лимиты и сетевые политики среды.	profile_id, cpu, ram, storage, network_policy	PostgreSQL
Knowledge Vector	Фрагменты базы знаний и контекст RAG.	collection, id, embedding, metadata, document	ChromaDB
Scenario Legend Vector	Векторные представления легенд, подсказок и подтвержденных решений.	scenario_id, embedding, domain, difficulty, source	ChromaDB
Session Status Cache	Оперативные статусы запуска и выполнения учебной среды.	key, session_id, status, ttl, updated_at	Redis
Task Queue Message	Очереди генерации, проверки и административных событий.	stream, message_id, type, payload, attempts	Redis

Hint Counter	Счетчики лимита подсказок в активной сессии.	`key`, `session_id`, `used_count`, `limit`, `ttl`	Redis

6. Архитектура системы в нотации C4

Архитектура проектируется как модульная сервисная система с четкими границами: клиентское веб-приложение, Backend/API, LangGraph-оркестратор, агенты, LLM Gateway, Sandbox Manager, Verification Engine, слой данных и наблюдаемость.

Контекстная диаграмма C4 приведена на Рис. 25. Она показывает пользователей, внешние системы и основные хранилища.

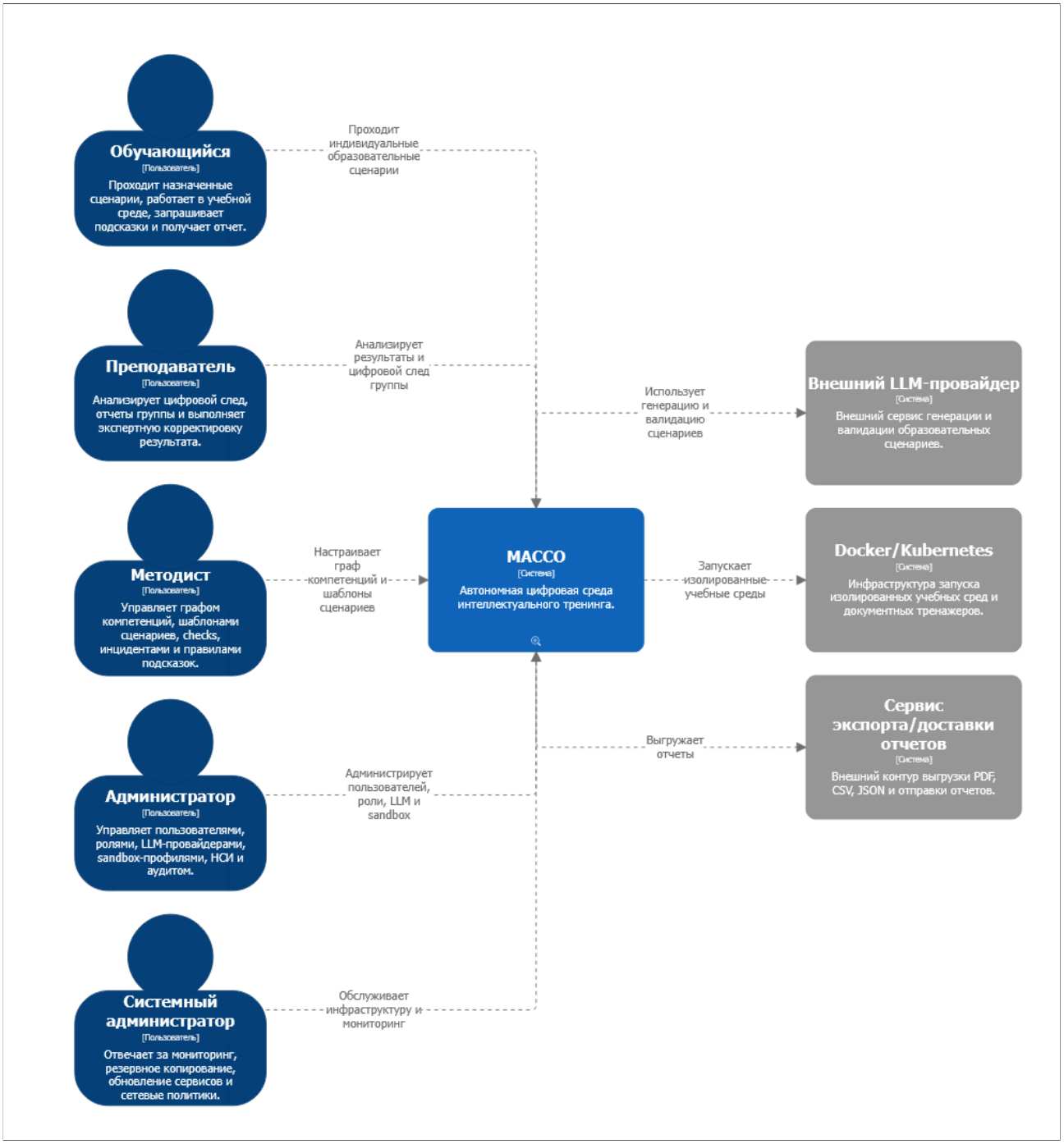


Рис. 25. C4 Context: MACCO во внешней среде

Диаграмма контейнеров приведена на Рис. 26. В ней закреплены контейнеры Frontend, Backend/API, Agent Workers, Sandbox Manager, Verification Engine и хранилища.

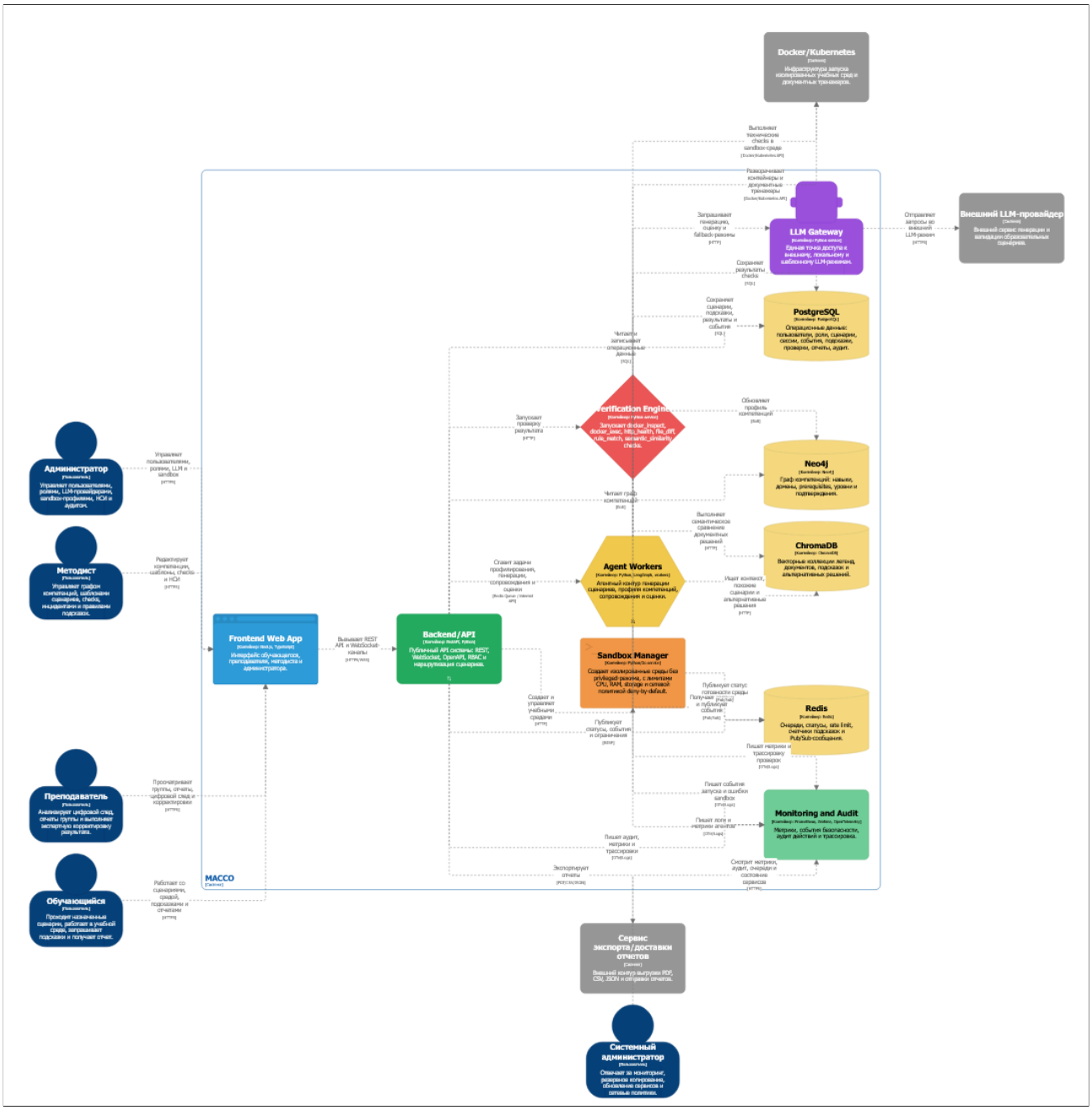


Рис. 26. C4 Container: контейнеры системы

Компонентная диаграммы backend и агентов приведены на Рис. 27, Рис. 28. Она показывает контроллеры API, сервисный слой, агенты и адаптеры к данным и инфраструктуре.

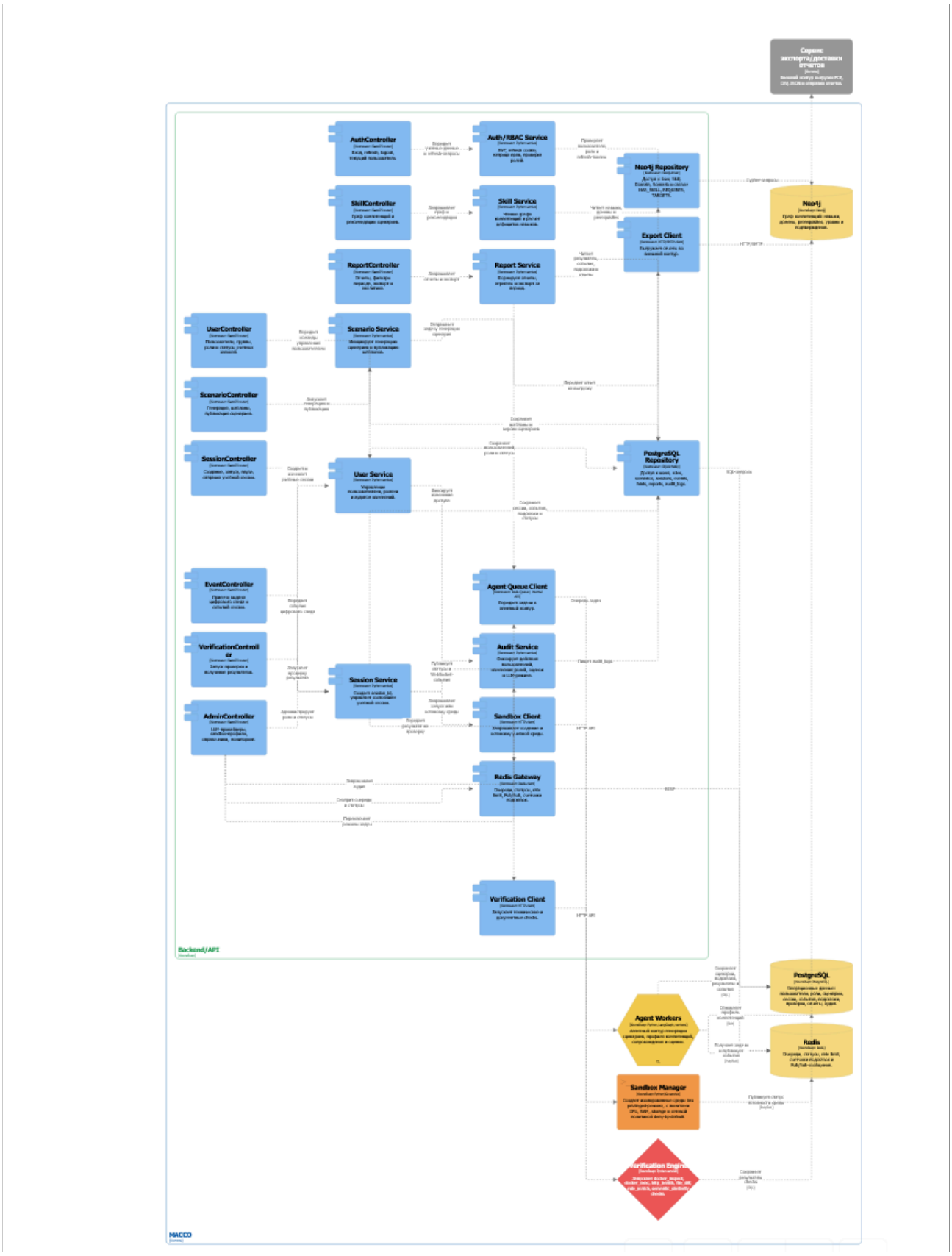


Рис. 27. C4 Component: backend и агенты

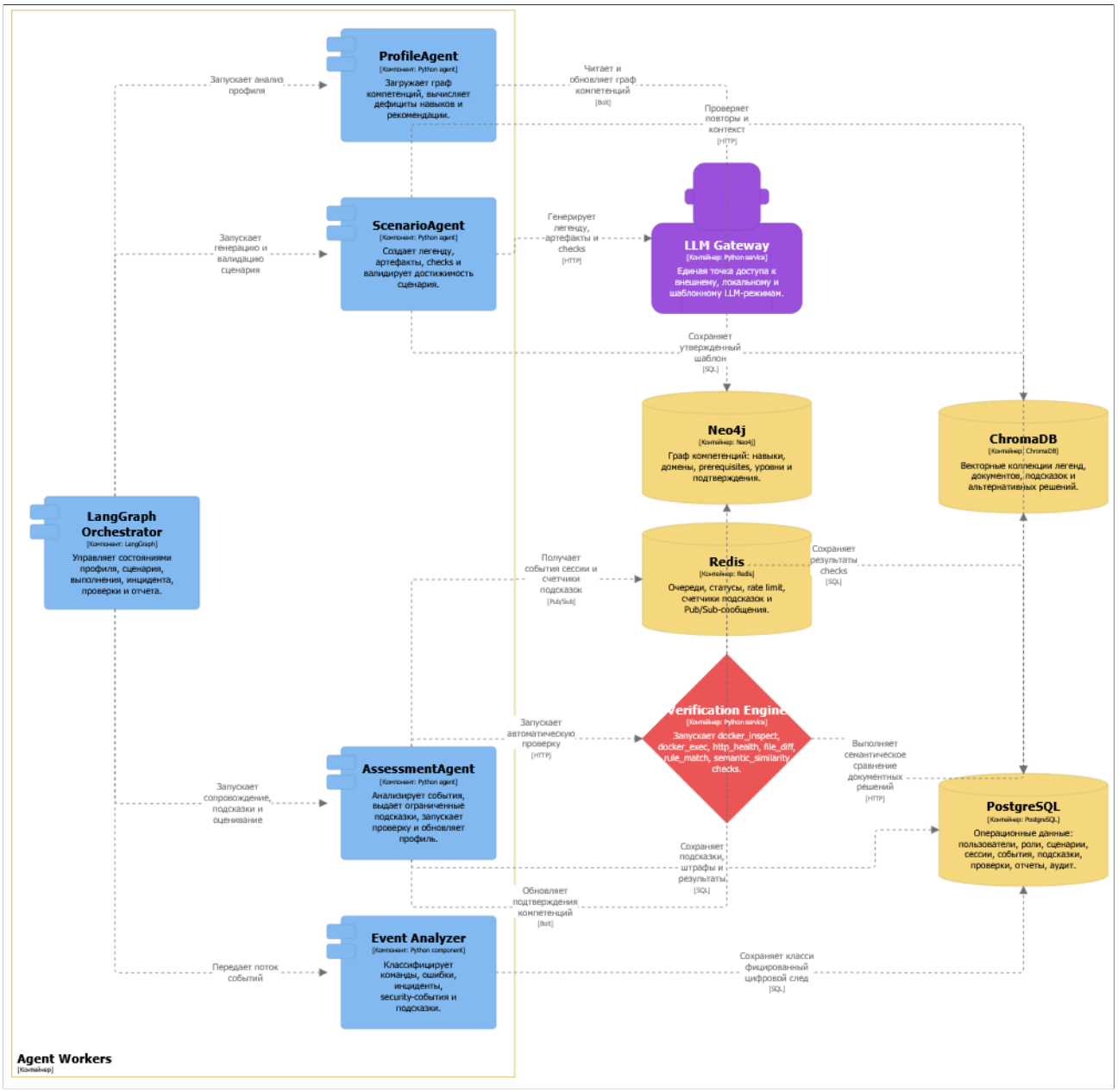


Рис. 28. C4 Component: backend и агенты

Диаграмма размещения приведена на Рис. 29. Она разделяет клиентскую зону, прикладную зону, sandbox-зону, контур данных и наблюдаемость.

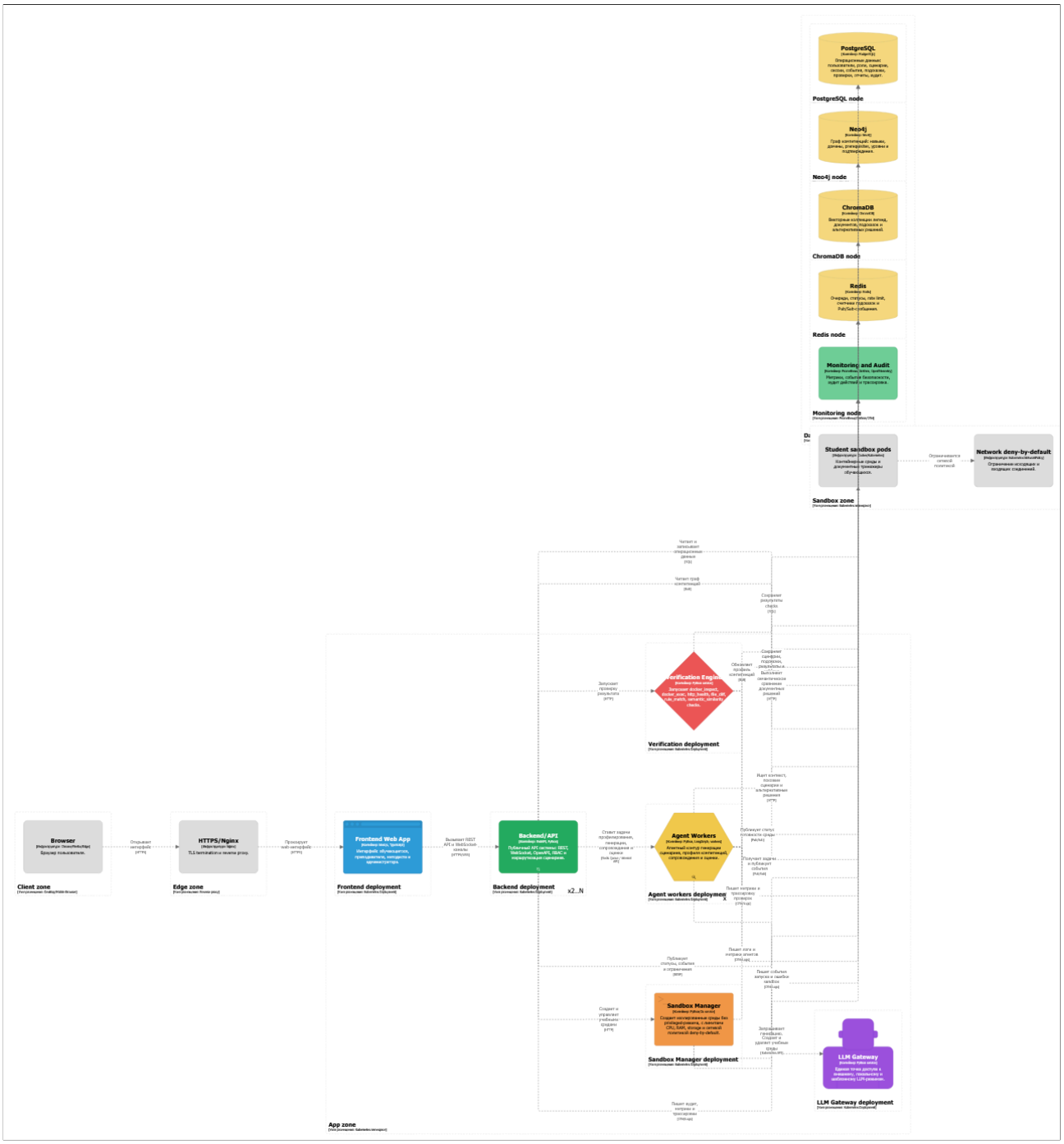


Рис. 29. C4 Deployment: размещение системы

Архитектурные решения.

- Frontend реализуется как Next.js-приложение с интерфейсами обучающегося, преподавателя, методиста и администратора.
- Backend/API реализуется на FastAPI и предоставляет REST, WebSocket и OpenAPI-спецификацию.
- LangGraph-оркестратор управляет состоянием многошагового процесса: профиль, сценарий, выполнение, инцидент, проверка, отчет.
- ProfileAgent отвечает за граф компетенций; ScenarioAgent - за генерацию и валидацию сценариев; AssessmentAgent - за подсказки, проверку и обновление профиля.

- LLM Gateway скрывает различия внешнего провайдера, локальной модели и шаблонного режима деградации.
- Sandbox Manager создает изолированные среды без привилегированного режима, с ограничениями CPU, RAM, storage и сетевой политикой deny-by-default.
- Verification Engine запускает проверки для контейнерных и документных сценариев.
- Monitoring and Audit собирает метрики, события безопасности, аудит действий и трассировку.

7. Модели информационных потоков данных

Модели информационных потоков выполнены в логике DFD. Они фиксируют внешние сущности, процессы, хранилища и направления движения данных.

Контекстная DFD приведена на Рис. 30. На ней система МАССО показана единым процессом, взаимодействующим с пользователями, LLM-провайдером, sandbox-инфраструктурой и хранилищами.

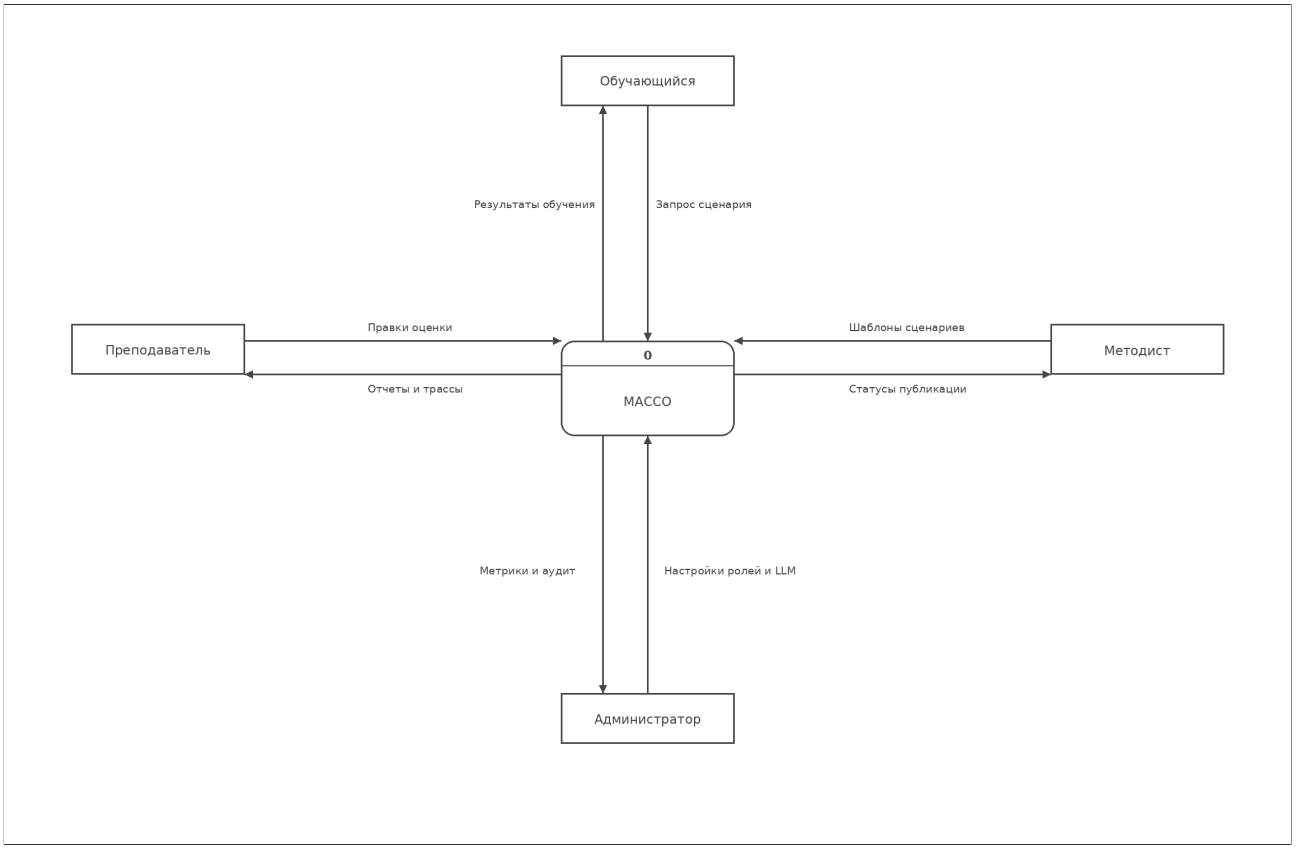


Рис. 30. Контекстная DFD МАССО

DFD уровня 1 для жизненного цикла обучения приведена на Рис. 31. Поток начинается с профиля компетенций, проходит генерацию сценария, выполнение задания и автоматическую проверку.

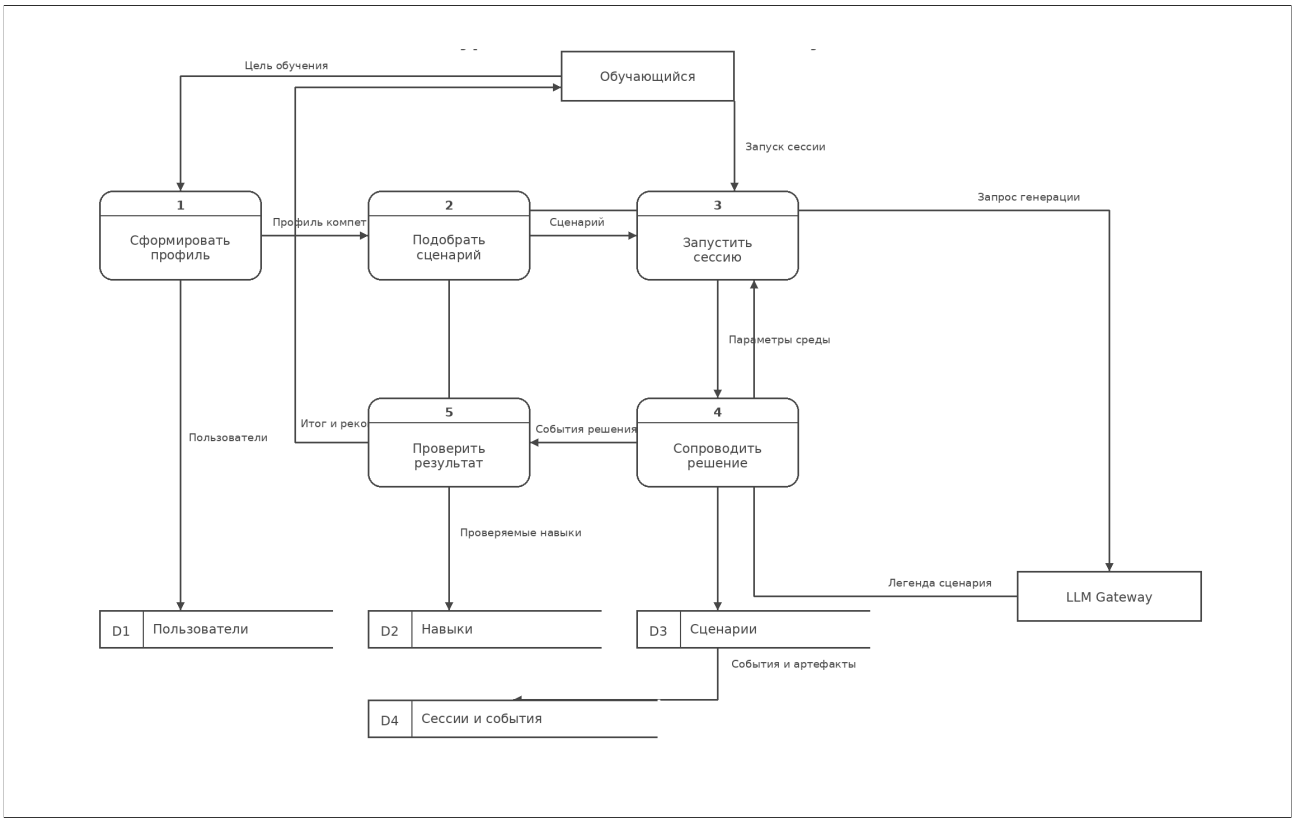


Рис. 31. DFD уровня 1: жизненный цикл обучения

DFD уровня 2 для отчетности и администрирования приведена на Рис. 32. На ней выделены фильтры периода, выгрузка отчетов, экспертная корректировка, настройка LLM и sandbox.

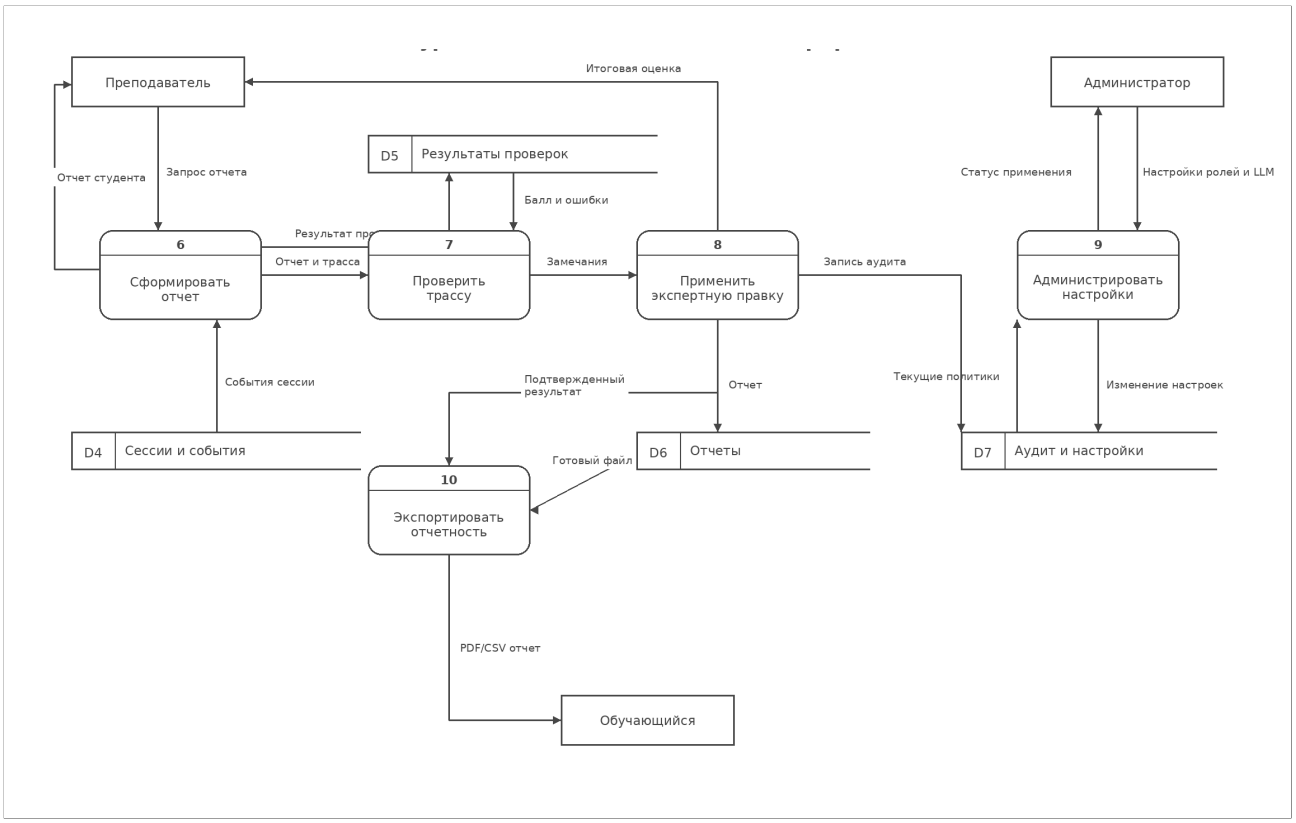


Рис. 32. DFD уровня 2: отчетность и администрирование

8. Шаблоны справочников и НСИ

Справочники используются при вводе и фильтрации информации, чтобы снизить число ошибок, обеспечить единообразие данных и выполнить требования методических рекомендаций к техническому проекту. Все справочники имеют поля `code`, `name`, `description`, `status`, `created_at`, `updated_at`; изменение справочников аудитируется.

Таблица 3 - Роли и права

Код	**Роль**	**Основные права**
`student`	Обучающийся	Запуск назначенных сценариев, работа в среде, запрос подсказок, отправка результата, просмотр своих отчетов.
`teacher`	Преподаватель	Просмотр групп, цифрового следа, отчетов, экспертное подтверждение или корректировка оценки.
`methodist`	Методист	Управление графом компетенций, шаблонами сценариев, checks, правилами подсказок.
`admin`	Администратор	Пользователи, роли, LLM-провайдеры, sandbox-профили, НСИ, аудит.
`sysadmin`	Системный администратор	Мониторинг инфраструктуры, резервное копирование, обновление сервисов, сетевые политики.

Таблица 4 - Предметные области и типы сценариев

Код	**Название**	**Пример использования**
`devops`	DevOps/администрирование	Восстановление веб-сервиса, диагностика Docker Compose, Nginx, БД.
`security`	Информационная безопасность	Закрытие лишнего порта, анализ подозрительной активности, ограничение прав.
`legal`	Юридический документный кейс	Поиск противоречий в договоре и учет новой нормативной вводной.
`audit`	Экономический или аудиторский кейс	Сверка первичных документов, обнаружение расхождения, расчет суммы.
`client_change`	Вводная от виртуального клиента	Изменение срока, бюджета, приоритета или дополнительного условия в ходе выполнения.

Таблица 5 - Статусы и события

Справочник	**Значения**	**Назначение**
Статусы навыка	`draft`, `active`, `deprecated`, `archived`	Управление жизненным циклом навыков в графе компетенций.
Статусы сценария	`draft`, `validating`, `published`, `archived`, `rejected`	Контроль публикации и версии шаблона сценария.
Статусы сессии	`created`, `starting`, `active`, `paused`, `submitted`, `checking`, `completed`, `failed`, `expired`	Отображение состояния учебной сессии.

Типы событий	command, file_change, answer, incident, hint, check, error, security	Классификация цифрового следа.
Severity	info, warning, error, critical	Фильтрация событий и сообщений в интерфейсе.

Таблица 6 - Проверки, подсказки, отчеты и инфраструктура

Справочник	**Значения**	**Использование**
Политики подсказок	none, limited_3, teacher_allowed	Ограничение числа подсказок и расчет штрафа.
Типы проверок	docker_inspect, docker_exec, http_health, file_diff, rule_match, semantic_similarity	Запуск Verification Engine.
Критерии оценки	required, weighted, critical, bonus	Расчет итогового балла и критических ошибок.
Форматы отчетов	pdf, csv, json	Экспорт отчетности с обязательными хронологическими фильтрами.
LLM-провайдеры	external, local, template	Выбор режима генерации и деградации.
Sandbox-профили	small, standard, secure, document	Лимиты CPU/RAM/storage и вид учебной среды.
Сетевые политики	deny_all, allowlist_http, internal_only, lab_network	Ограничение сетевого доступа сессий.

9. Проектирование API

Публичный API проектируется в REST-стиле. Все ответы возвращаются в JSON и содержат `request_id`, `status`, `data` или `error`. Для терминала, статусов сессии и журнала событий используется WebSocket с краткоживущим токеном.

Таблица 7 - Общий формат REST-ответа

Поле	Тип	Описание
`request_id`	`string`	Корреляционный идентификатор запроса.
`status`	`success | error`	Результат обработки.
`data`	`object | array | null`	Полезная нагрузка успешного ответа.
`error.code`	`string`	Машинно-читаемый код ошибки.
`error.message`	`string`	Понятное сообщение на русском языке.
`error.details`	`object`	Дополнительные сведения без раскрытия секретов.

Таблица 8 - REST-контуры API

Контур	Назначение	Операции
`/auth`	Аутентификация и сессии доступа.	`POST /login`, `POST /refresh`, `POST /logout`, `GET /me`.
`/users`	Пользователи, группы, роли.	`GET /users`, `POST /users`, `PATCH /users/{id}`, `PUT /users/{id}/roles`.
`/skills`	Граф компетенций и уровни навыков.	`GET /skills/graph`, `POST /skills`, `PATCH /skills/{id}`, `GET /skills/recommendations`.
`/scenarios`	Шаблоны и сгенерированные сценарии.	`POST /generate`, `POST /templates`, `PATCH /templates/{id}`, `POST /templates/{id}/publish`.
`/sessions`	Учебные сессии.	`POST /sessions`, `GET /sessions/{id}`, `POST /sessions/{id}/pause`, `POST /sessions/{id}/submit`.
`/events`	Цифровой след и события.	`GET /events?session_id`, `POST /events`, `GET /events/export`.
`/verification`	Проверка и экспертная оценка.	`POST /verification/run`, `GET /verification/{id}`, `PATCH /verification/{id}`.
`/reports`	Отчеты и аналитика.	`GET /reports?from&to`, `GET /reports/{id}`, `GET /reports/export?format`.
`/admin/llm`	LLM-провайдеры и режимы.	`GET /providers`, `PATCH /providers/{id}`, `POST /switch-mode`.
`/admin/sandbox`	Профили песочниц и политики сети.	`GET /profiles`, `POST /profiles`, `PATCH /profiles/{id}`, `GET /health`.

Таблица 9 - WebSocket-каналы

Канал	События	Назначение
`/ws/sessions/{id}/terminal`	`stdin`, `stdout`, `stderr`, `resize`, `close`	Интерактивный терминал контейнерной среды.
`/ws/sessions/{id}/events`	`incident`, `hint`, `warning`, `security`, `check_status`	Поток событий учебной сессии.

/ws/sessions/{id}/status	starting, ready, paused, checking, completed, failed	Состояние среды и проверки.
/ws/admin/monitoring	queue, provider, sandbox, alert	Административные статусы LLM, очередей и sandbox.

Основные ошибки API.

Система должна возвращать единые коды ошибок: AUTH_INVALID_CREDENTIALS, AUTH_FORBIDDEN, SCENARIO_NOT_VALID, SESSION_NOT_READY, HINT_LIMIT_EXCEEDED, VERIFICATION_FAILED, LLM_PROVIDER_UNAVAILABLE, SANDBOX_LIMIT_EXCEEDED, REPORT_PERIOD_REQUIRED, VALIDATION_ERROR.

10. Модель структуры интерфейса

Модель структуры интерфейса показана на Рис. 33. Она разделяет общие экраны и рабочие области по ролям: обучающийся, преподаватель, методист, администратор.

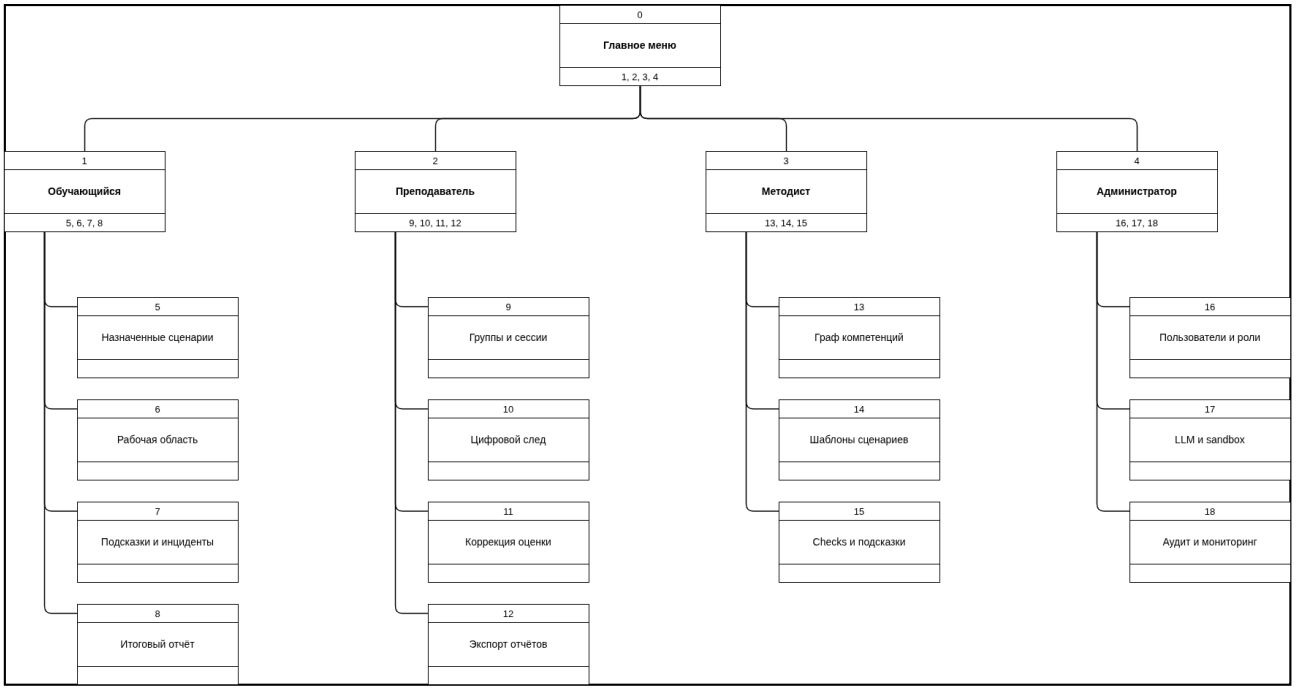


Рис. 33. Модель структуры интерфейса системы

Обязательные интерфейсные решения.

- Метафора интерфейса: учебная диспетчерская, где сценарий выглядит как назначение, рабочая среда - как рабочее место, а отчет - как протокол прохождения.
- Цветовая схема: нейтральная основа, синий для рабочих действий, зеленый для успешных проверок, желтый для предупреждений, красный для ошибок и событий безопасности.
- При вводе информации используются справочники: роли, группы, предметные области, типы сценариев, статусы, severity, форматы отчетов, LLM-режимы, sandbox-профили.
- Сообщения состояния обязательны для генерации, запуска среды, ожидания проверки, экспорта отчета и переключения LLM-контура.
- Сообщения об ошибках должны быть понятны пользователю и не раскрывать системные промпты, секреты провайдеров, внутренние checks и пути хостовой файловой системы.
- В формах ввода сохраняется пропорция: подпись занимает около 30-35 % ширины строки, поле ввода - 65-70 %.
- Отчетность всегда содержит хронологические фильтры `Дата начала` и `Дата окончания`; без периода экспорт не запускается.

11. Макеты интерфейсов системы

Макеты являются проектными HTML-экранами, а не готовой реализацией. Они показывают состав элементов, сообщения состояния, фильтры, справочники и основные действия пользователя. В отчет вставлены изображения, сформированные из страниц каталога `mockups_html`.

Экран входа показан на Рис. 34. Он содержит поля учетных данных, кнопку входа, состояние проверки и сообщение об ошибке.

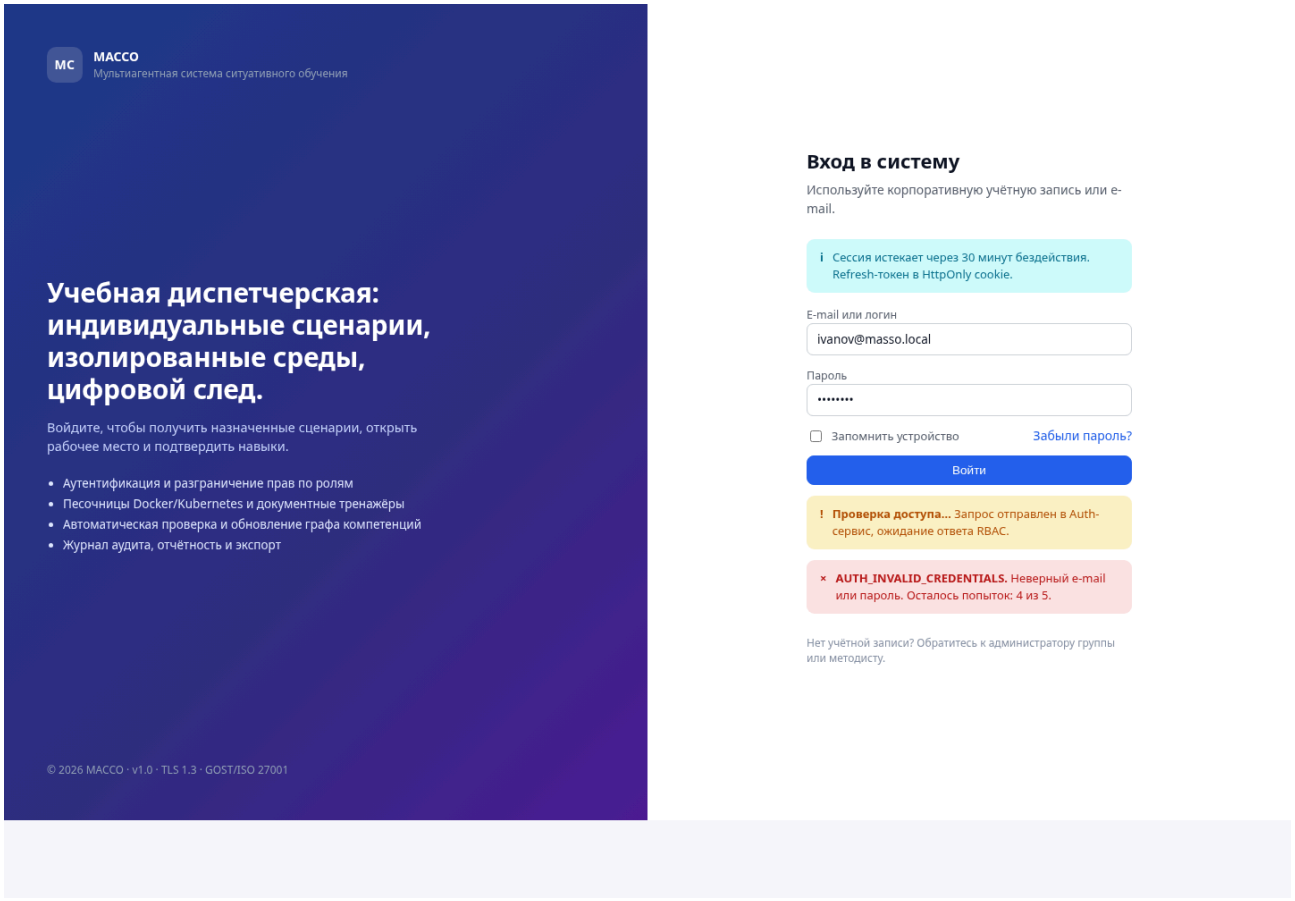


Рис. 34. Макет интерфейса входа в систему

Панель обучающегося показана на Рис. 35. В ней отображаются назначенные сценарии, рекомендации и отчеты.

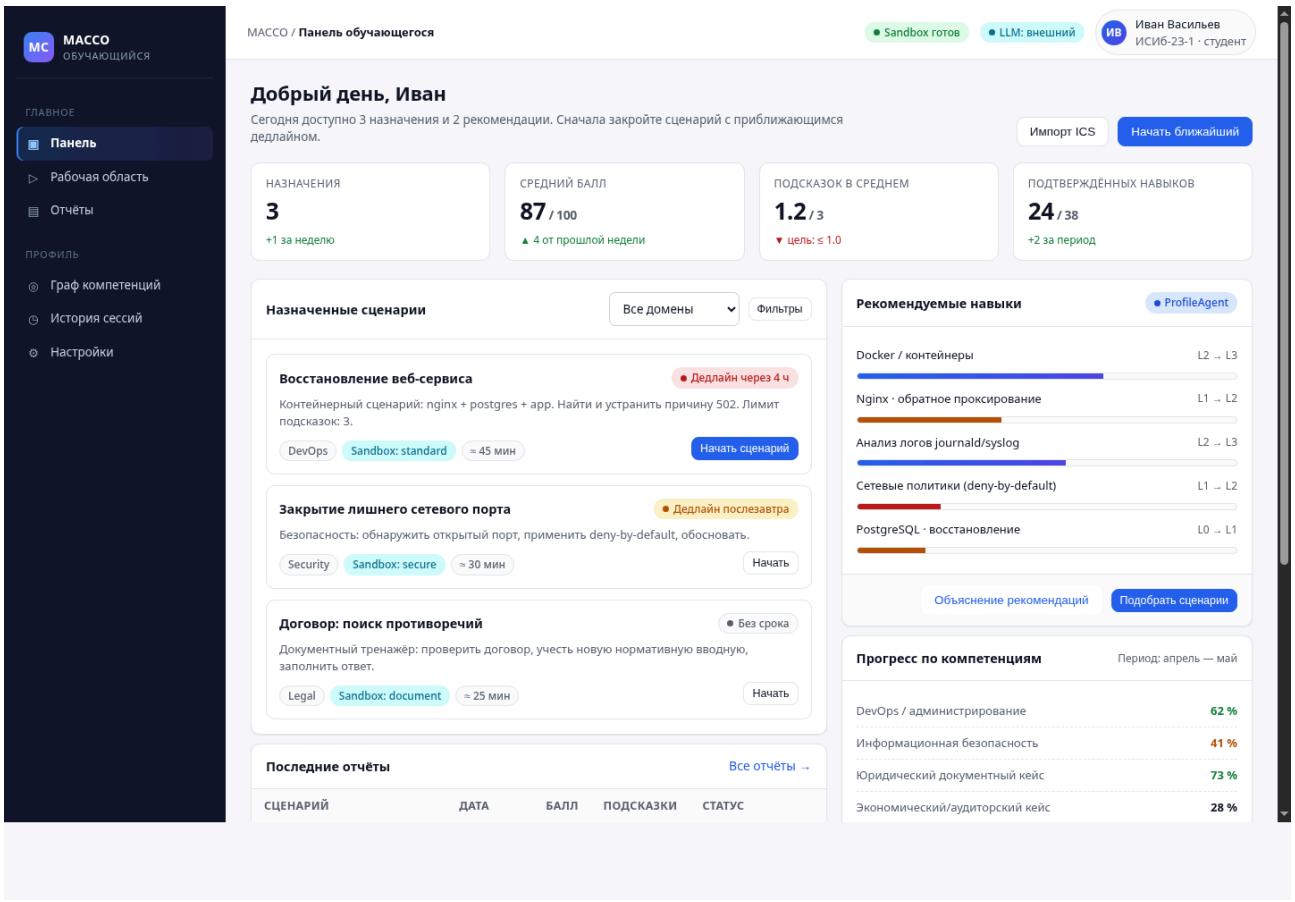


Рис. 35. Макет панели обучающегося

Рабочая область обучающегося показана на Рис. 36. Она объединяет легенду, терминал или документный тренажер, журнал инцидентов, подсказки и отправку результата.

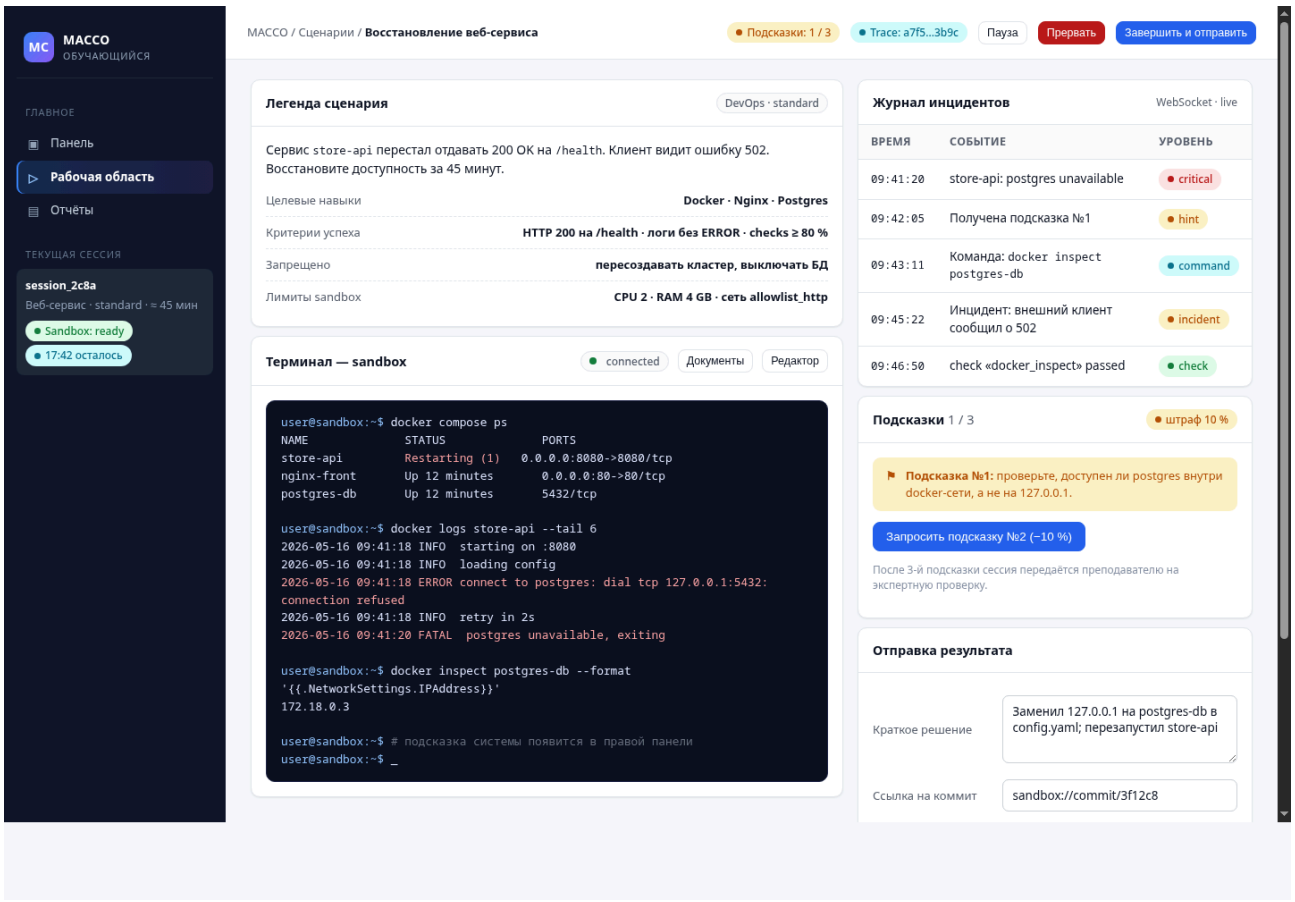


Рис. 36. Макет рабочей области обучающегося

Итоговый отчет обучающегося показан на Рис. 37. На экране выводятся балл, checks, ошибки, подсказки и рекомендации.

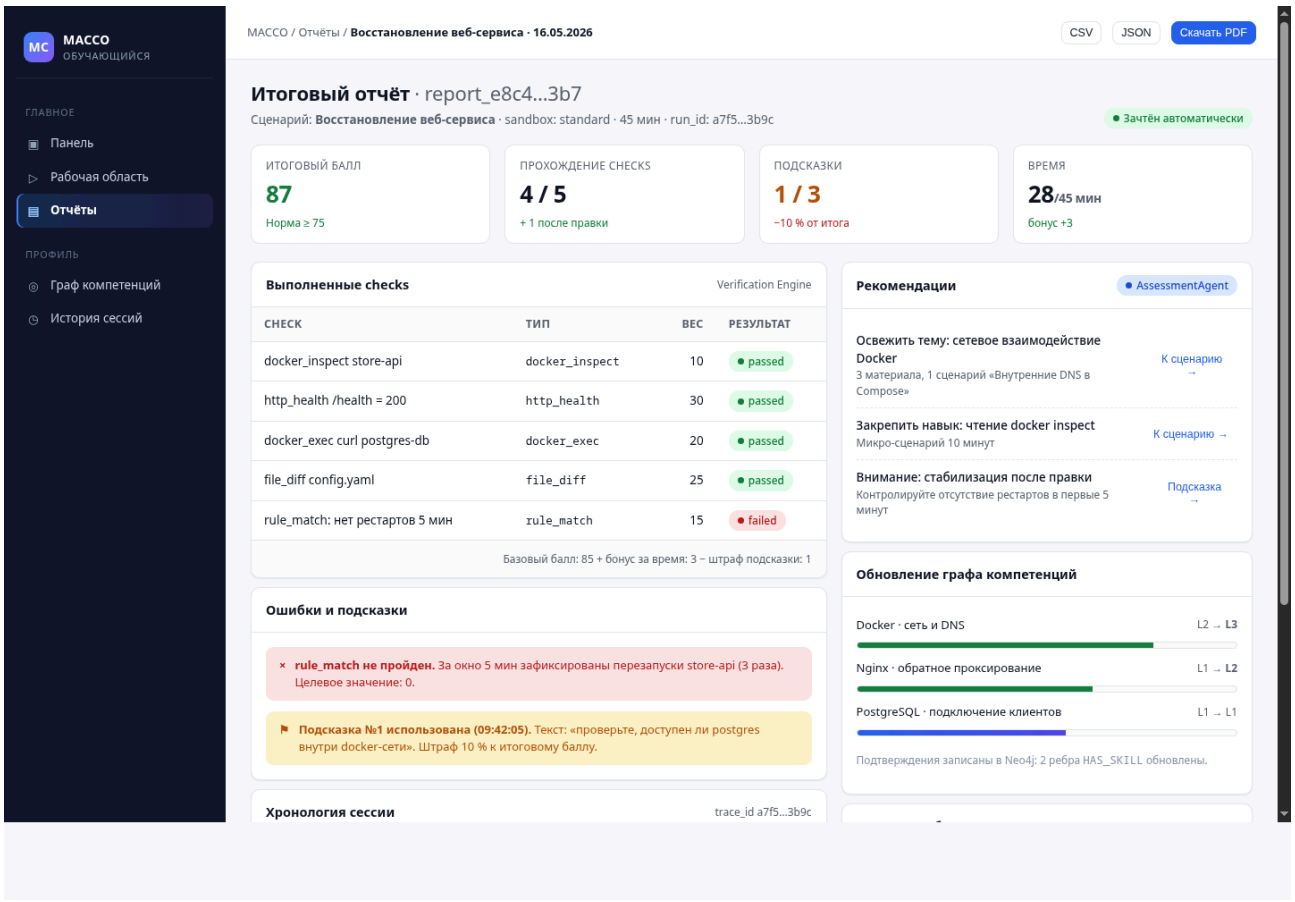


Рис. 37. Макет итогового отчета обучающегося

Панель преподавателя показана на Рис. 38. Обязательными являются фильтры группы и периода, агрегаты, проблемные навыки и список сессий.

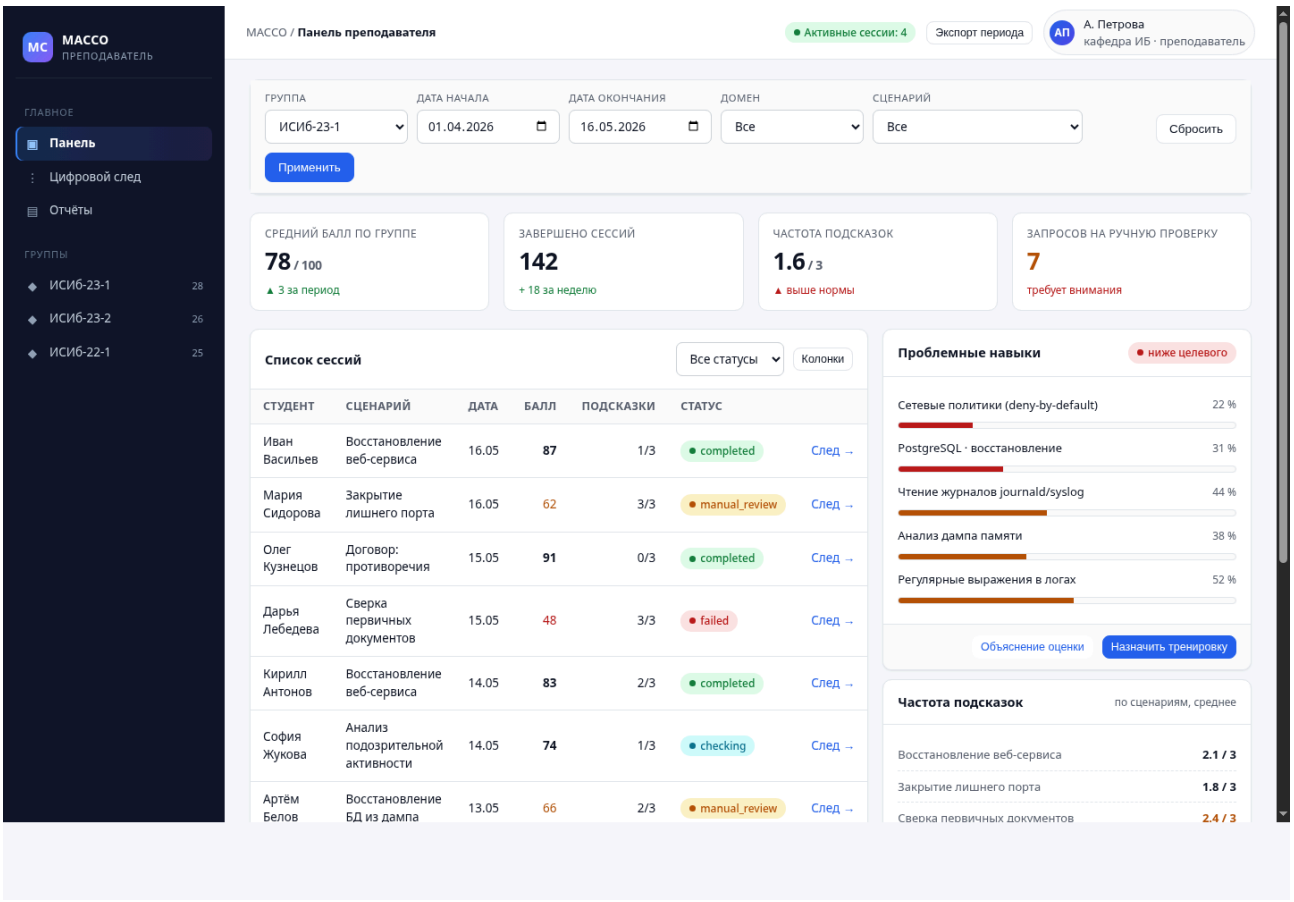


Рис. 38. Макет панели преподавателя

Экран цифрового следа показан на Рис. 39. Он позволяет фильтровать события, анализировать проверки и корректировать оценку с комментарием.

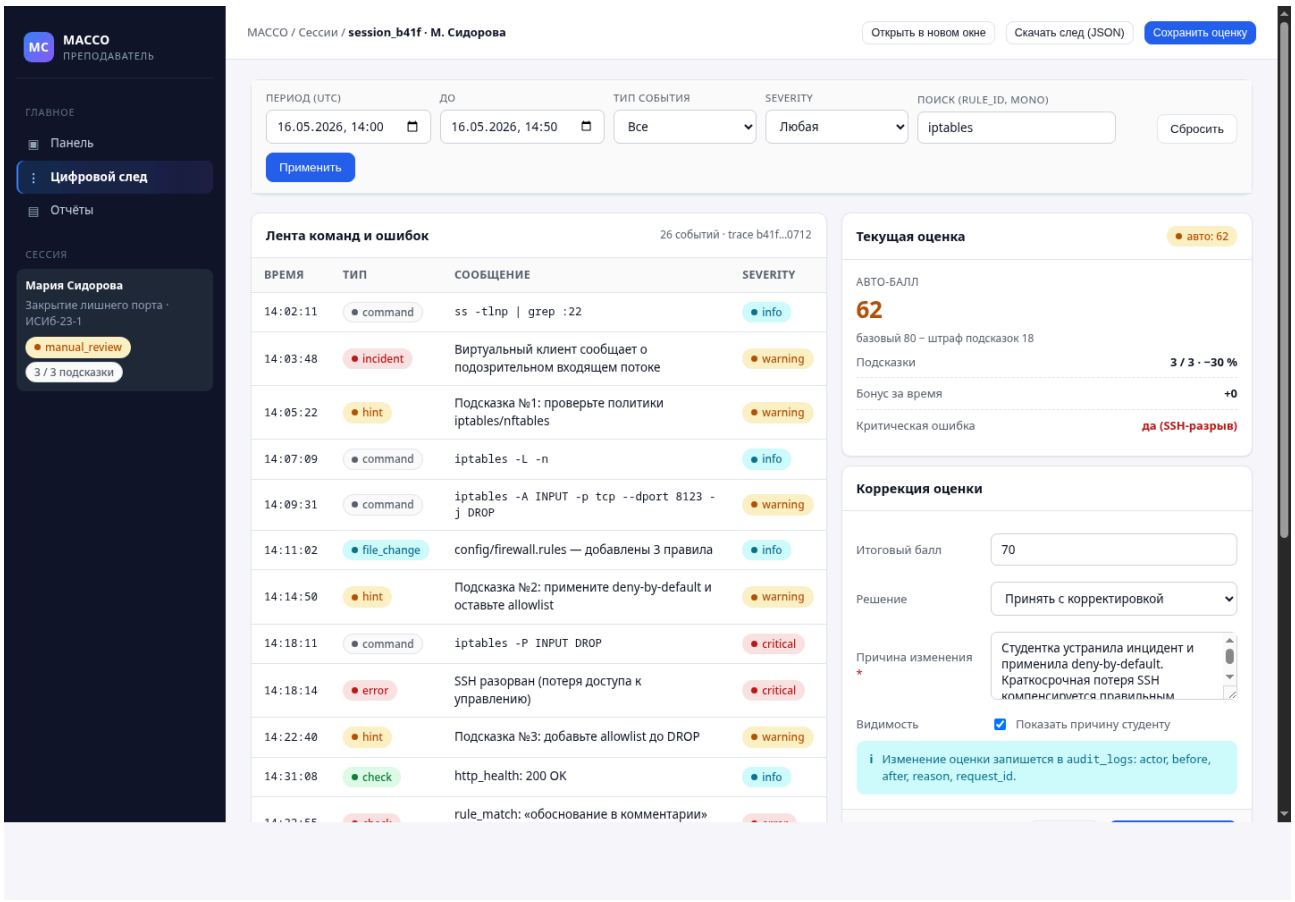


Рис. 39. Макет просмотра цифрового следа преподавателем

Экран графа компетенций методиста показан на Рис. 40. Он содержит дерево доменов, граф навыков, свойства узла и импорт CSV/JSON.

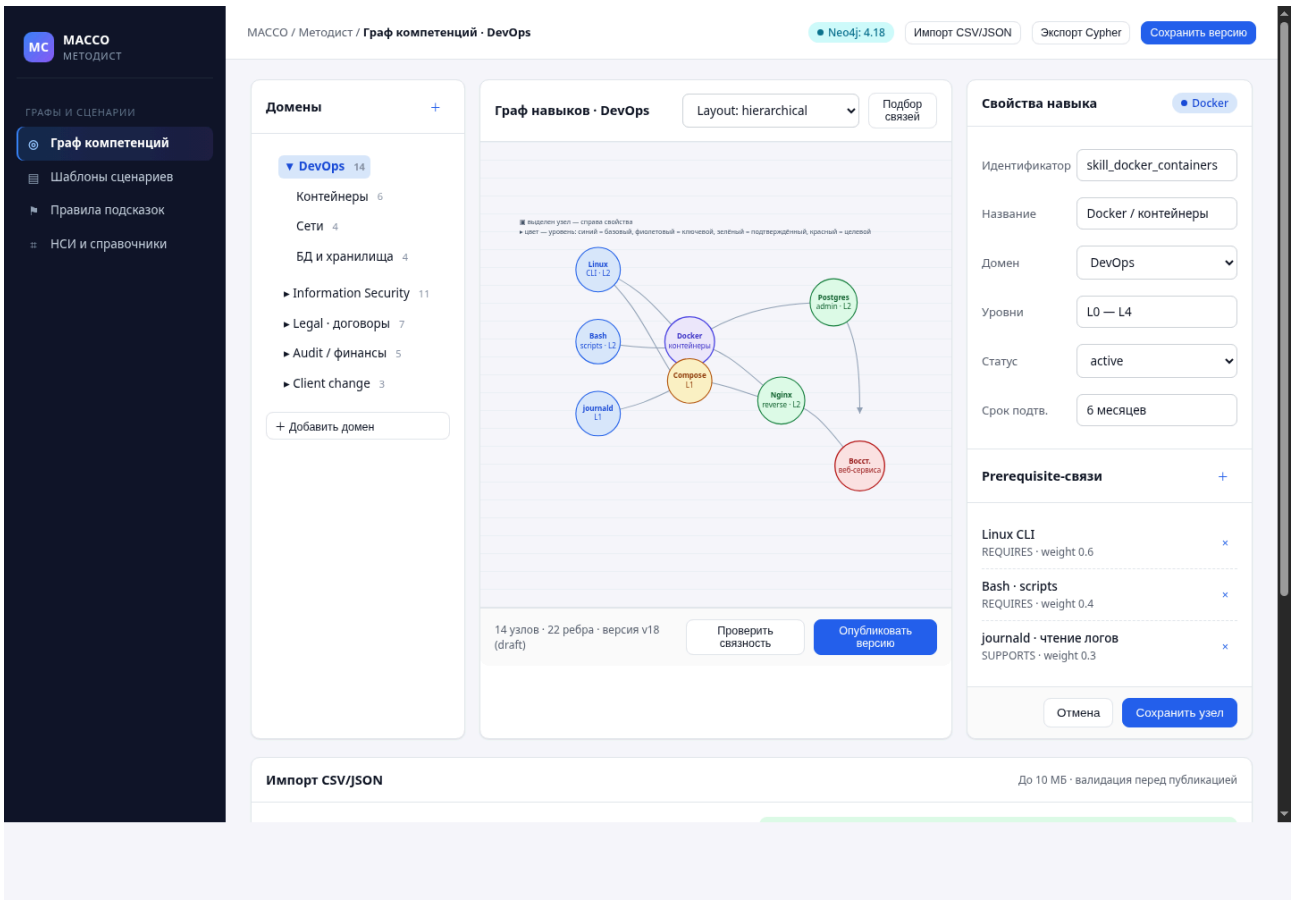


Рис. 40. Макет управления графом компетенций

Экран шаблона сценария показан на Рис. 41. Методист управляет легендой, целевыми навыками, checks, инцидентами и подсказками.

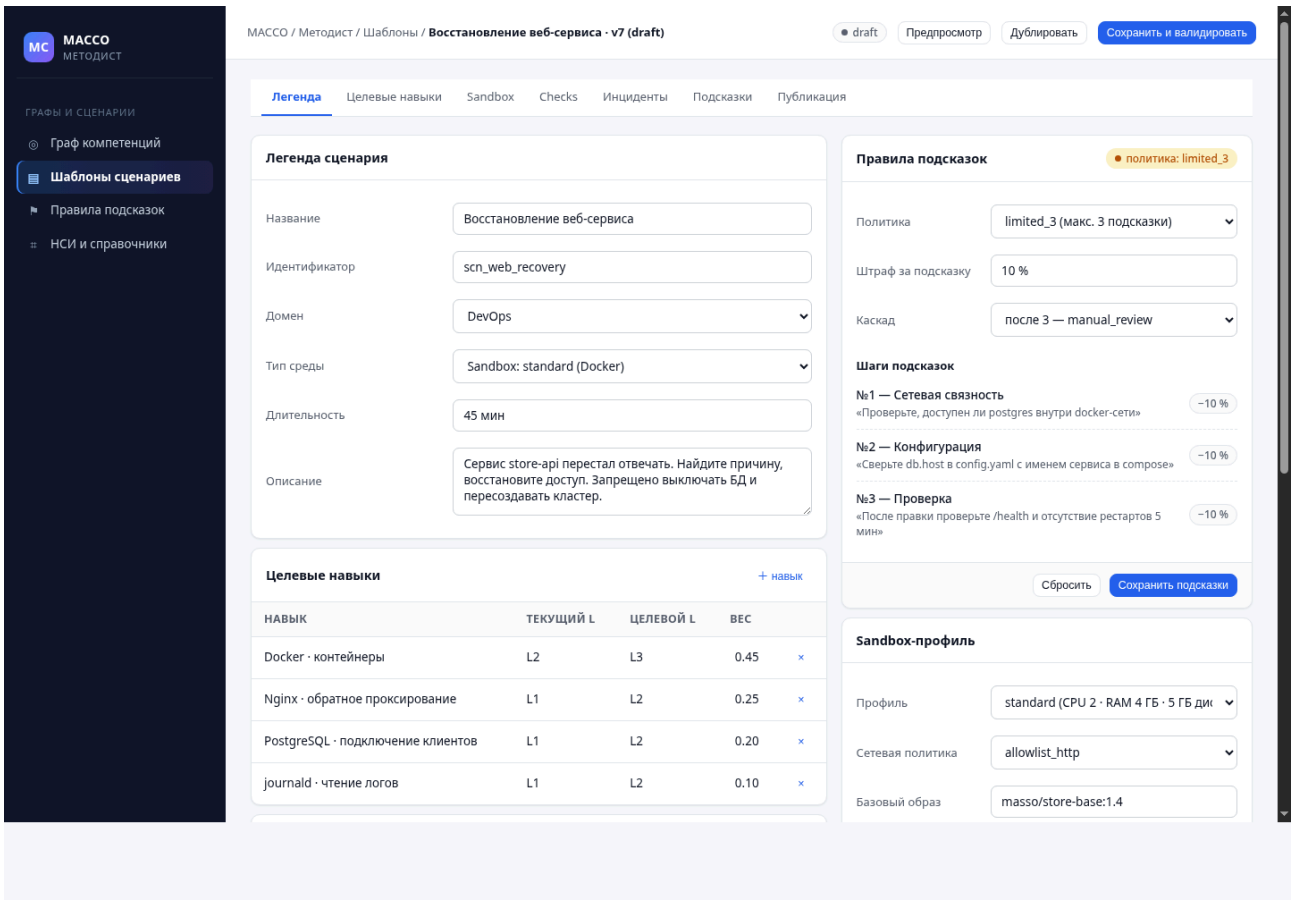


Рис. 41. Макет редактирования шаблона сценария

Экран пользователей и ролей администратора показан на Рис. 42. Он поддерживает поиск, изменение ролей, статуса учетной записи и аудит изменений.

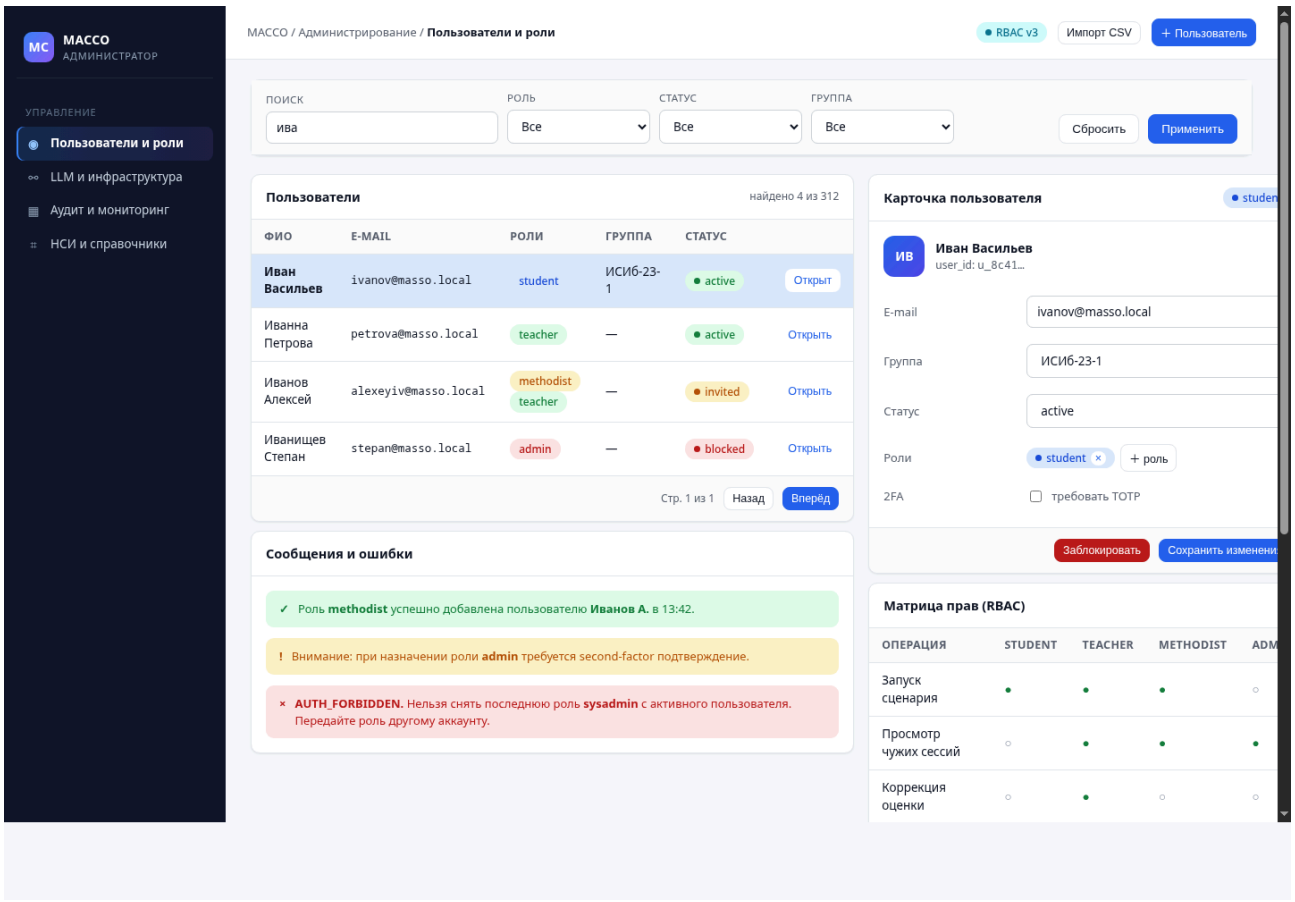


Рис. 42. Макет управления пользователями и ролями

Экран LLM и инфраструктуры показан на Рис. 43. Администратор видит режим провайдеров, очереди, sandbox-профили и состояние системы.

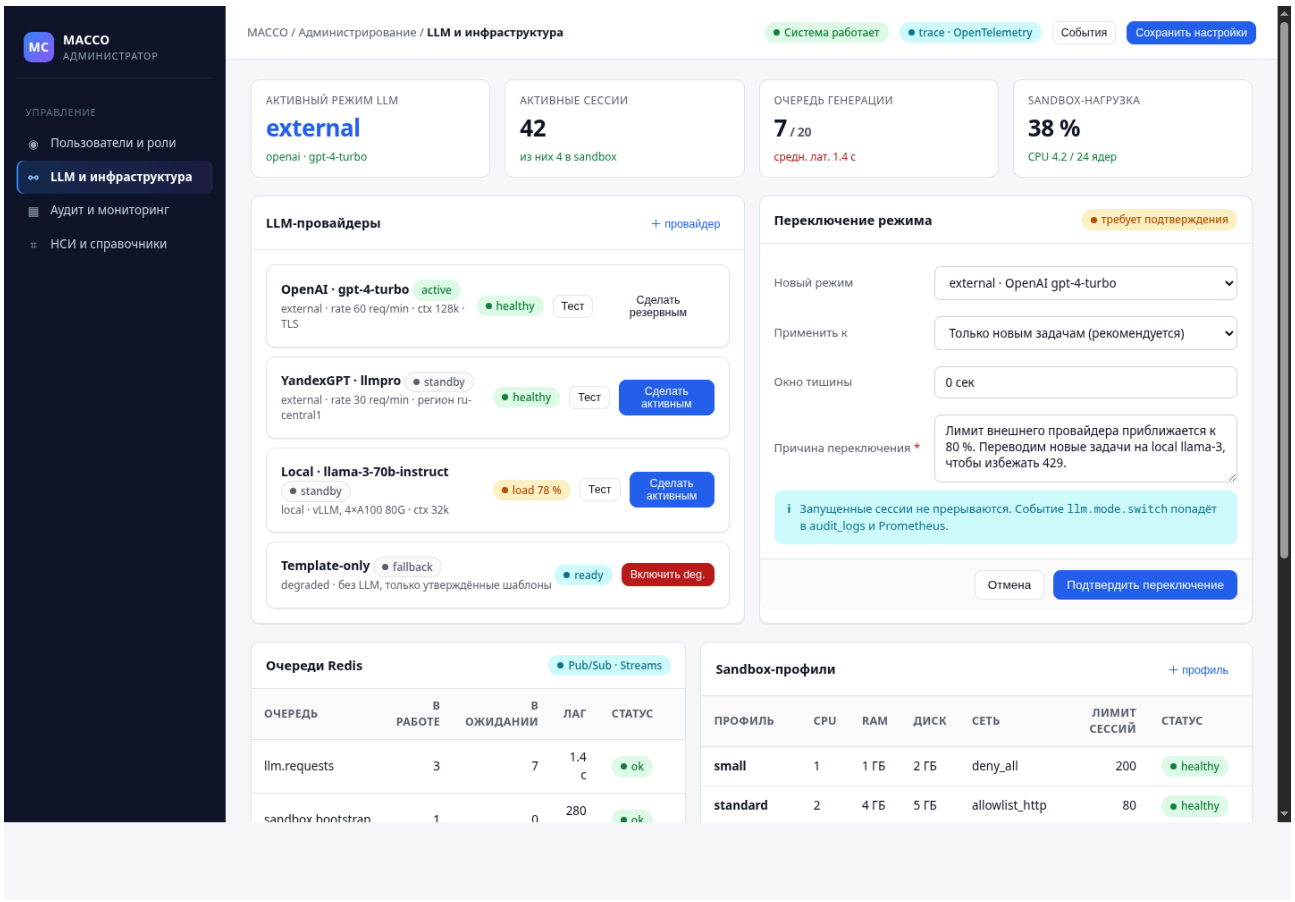


Рис. 43. Макет администрирования LLM и инфраструктуры

Экран аудита и мониторинга показан на Рис. 44. Он содержит периодные фильтры, события безопасности, метрики и экспорт журнала.

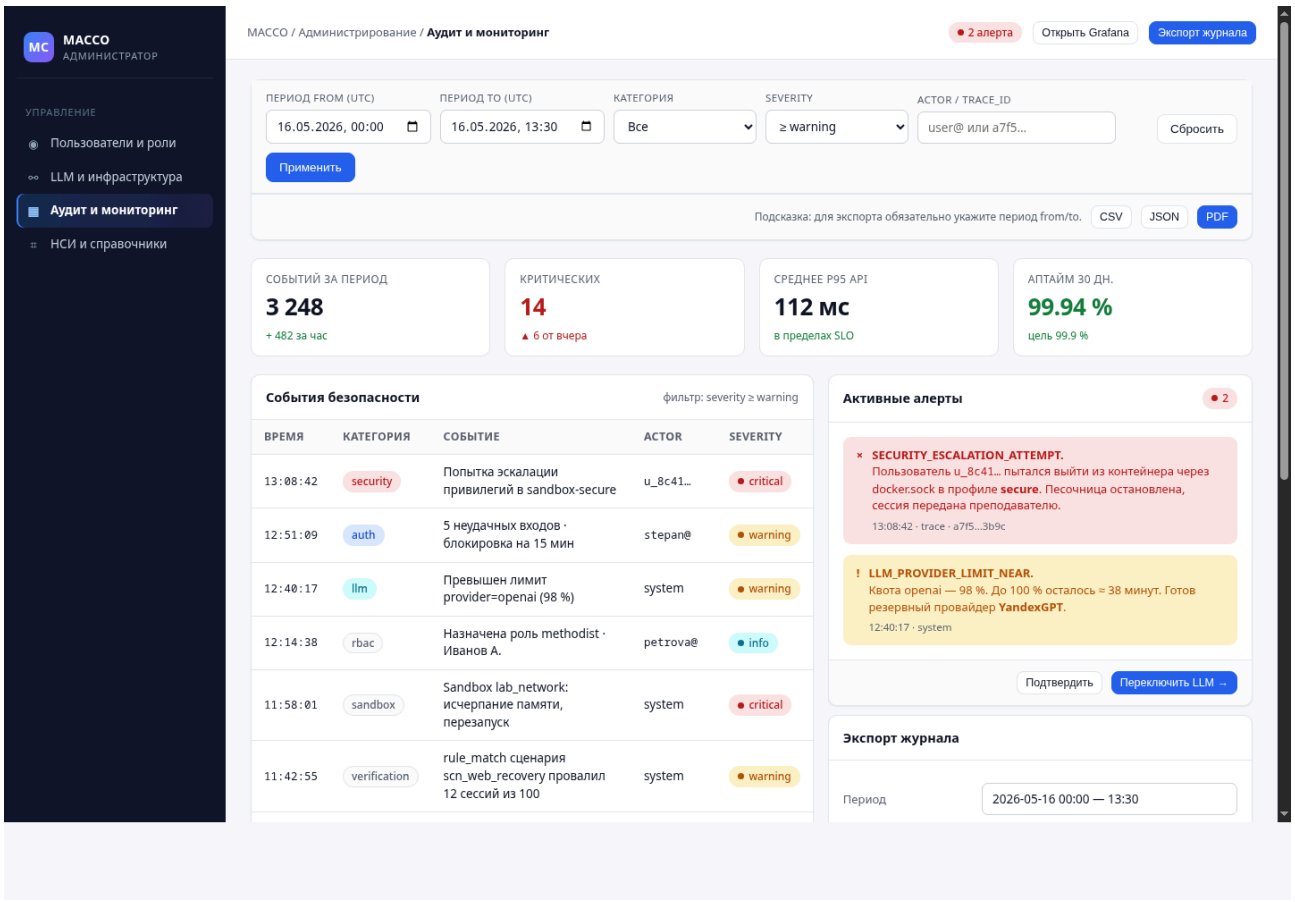


Рис. 44. Макет аудита и мониторинга

12. Контрольные проектные сценарии

Сценарий 1. Обучающийся проходит DevOps-сценарий.

Обучающийся входит в систему, выбирает назначение восстановления веб-сервиса, запускает среду, анализирует контейнеры, исправляет ошибку БД и Nginx, отправляет результат. Система фиксирует команды, инцидент, подсказки, запускает checks, формирует отчет и обновляет граф компетенций.

Сценарий 2. Обучающийся проходит юридический документный кейс.

Обучающийся открывает пакет документов, находит противоречия, учитывает новую нормативную вводную, заполняет форму ответа. AssessmentAgent выполняет правило проверки и семантическое сравнение.

Сценарий 3. Преподаватель анализирует результаты группы.

Преподаватель выбирает группу и период, просматривает средний балл, частоту подсказок, проблемные навыки и сессии с расхождениями. При корректировке результата причина сохраняется в журнале аудита.

Сценарий 4. Методист добавляет шаблон сценария.

Методист добавляет навык в граф, указывает prerequisite-связи, задает checks, инциденты, лимит времени и правила подсказок. Сценарий публикуется только после валидации.

Сценарий 5. Администратор переключает LLM-контур.

Администратор получает предупреждение о лимите внешнего провайдера и переводит новые генерации в локальный или шаблонный режим. Запущенные сессии продолжают работу, переключение аудитируется.

13. Вывод

В техническом проекте выполнена детализация системы МАССО на основе технического задания: определены сущности, процессы, последовательности взаимодействия, модели данных для всех хранилищ, архитектура C4, информационные потоки, справочники, API, структура интерфейса и макеты экранов. Диаграммные артефакты размещены в каталоге `images`, а интерфейсные макеты представлены HTML-страницами в каталоге `mockups_html` и вставлены в документ как изображения с описанием и ссылками.

СОСТАВИЛ

Наименование организации, предприятия	Должность исполнителя	Фамилия, имя, отчество	Подпись	Дата
ИРНИТУ	Студент гр. ИСИб-23-1	Л.А. Демонов		

СОГЛАСОВАНО

Наименование организации, предприятия	Должность исполнителя	Фамилия, имя, отчество	Подпись	Дата
ИРНИТУ	Руководитель	Д. С. Шипицин		